

Guía para culminar el Proyecto Fin de Curso

Equipo ACC

Aplicaciones Distribuidas (ISR-701) · Séptimo semestre · Carrera de Ingeniería de Software
Facultad de Ciencias de la Computación y Diseño Digital · UTEQ · PPA 2026–2027

El proyecto

TicketFold — Soporte técnico distribuido para un proveedor de Internet

Repositorio: <https://github.com/carlospatroner-boop/PFC-SOPORTE-ISP>

Estado revisado sobre la rama main, *commit* 964fa6e del 13 de septiembre de 2026, con 187 *commits* de historia.

Integrantes: Cristhian Pacheco, Carlos Carpio, Robinson Cando, Jeremy Álvarez.

1 Para qué sirve esta guía

Esta guía responde a una sola pregunta: qué les falta para cerrar el Proyecto Fin de Curso con un manuscrito que se sostenga. Para responderla revisé el repositorio entero sobre el último *commit*, entregable por entregable, sin límite de fecha. No me quedé en comprobar que los archivos estuvieran: ejecuté las suites de pruebas, medí yo mismo la cobertura sin los filtros del proyecto, recalculé desde los datos crudos las cifras que el documento publica, ejecuté las verificaciones de sumas contando cuántas pasan y cuántas fallan, y comprobé en cada caso que las pruebas pudieran fallar rompiendo una aserción o el dato.

El resultado es la tabla de la sección 3: los **44 entregables** que el proyecto debía desarrollar, cada uno con su estado. De ellos, 31 están en Hecho y 13 necesitan trabajo. Ese es el alcance real de lo que queda.

Lectura del estado del proyecto

Este es el proyecto que llega en mejor estado al cierre: la medición es limpia, los datos crudos aguantan el recálculo, las verificaciones de sumas pasan 45 de 45 y corren en el flujo automático, y el generador del reporte de correlación regenera la tabla del documento byte a byte. Lo que queda pendiente es de arquitectura y de cierre, no de medición. El dominio que el documento declara libre de marco importa el marco en diez clases, la base de datos no tiene motor de migraciones, la aplicación móvil captura fotografía y coordenadas que nunca salen del teléfono, la compuerta de cobertura está escrita de forma que no puede fallar, y los conteos de pruebas del documento no se reproducen.

2 Los cuatro estados

Estado	Qué significa exactamente
Hecho	El entregable está terminado, es correcto al cien por cien y está listo para usarse en la redacción del manuscrito. Se puede citar tal como está: la cifra que publica se reproduce, el archivo que nombra existe, la prueba que declara puede fallar.
Modificar	El entregable está terminado, pero no está correctamente desarrollado. No está listo para la redacción: usarlo tal como está introduciría en el manuscrito un error o una afirmación que el repositorio no sostiene.
Culminar	El desarrollo está iniciado y hay que continuarlo. La pieza existe y funciona en parte, pero le falta lo que la haría utilizable como evidencia.
Iniciar	El entregable no está presente en el proyecto. Hay que construirlo desde cero.

La diferencia entre Modificar y Culminar, que es la que más se confunde

Un entregable en **Modificar** ya está construido y el trabajo que queda es de corrección: cambiar un número que no cuadra, quitar una afirmación que el código no sostiene, retirar un filtro que infla una medición. Suele costar poco tiempo y suele ser lo que más daño hace si se deja, porque un revisor externo lo detecta abriendo el repositorio y pierde la confianza en todo lo demás.

Un entregable en **Culminar** tiene el trabajo a medias y lo que queda es construcción: ejecutar una campaña que no se ejecutó, escribir el verificador que falta, capturar la evidencia que nadie capturó. Cuesta más tiempo, pero mientras está a medias no engaña a nadie.

Por eso el orden de ataque que propongo más adelante empieza por lo que está en Modificar y no por lo que parece más grande.

3 Todos los entregables, con su estado

La tabla recoge todo lo que el proyecto debía desarrollar, incluido lo que ya está resuelto. La columna de estado es la que manda: lo que aparece en Hecho no hay que volver a tocarlo, y puede citarse en el manuscrito tal como está.

#	Entregable	Ámbito	Estado
1	Refactorización en capas y principios SOLID	Arquitectura	Modificar
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho
3	Vistas de arquitectura C4 (tres niveles)	Arquitectura	Culminar
4	Contrato de interfaz OpenAPI	Arquitectura	Culminar
5	Esquema de base de datos y migraciones	Datos	Iniciar
6	Persistencia distribuida operando	Datos	Hecho
7	Aplicación web funcional (construye y ejecuta)	Aplicaciones	Hecho
8	Calidad de la aplicación web (pruebas de unidad)	Pruebas	Hecho
9	Aplicación móvil funcional	Aplicaciones	Hecho
10	Capacidades de dispositivo (cámara y localización)	Aplicaciones	Culminar
11	Integración web + móvil + backend	Aplicaciones	Culminar
12	Pruebas de unidad e integración del backend	Pruebas	Culminar
13	Pruebas de contrato	Pruebas	Hecho
14	Pruebas de extremo a extremo	Pruebas	Hecho
15	Pruebas de carga	Pruebas	Hecho

#	Entregable	Ámbito	Estado
16	Pipeline de integración y despliegue con publicación etiquetada	Flujo	Culminar
17	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Hecho
18	Procesamiento paralelo	Medición	Hecho
19	Módulo de correlación implementado y probado	Medición	Hecho
20	Protocolo experimental y campaña ejecutada según diseño	Medición	Hecho
21	Análisis estadístico con prueba no paramétrica y tamaño de efecto	Medición	Hecho
22	Correspondencia exacta entre el análisis y las tablas del documento	Manuscrito	Modificar
23	Datos crudos preservados	Medición	Hecho
24	Evaluación de calidad con las características medidas	Medición	Hecho
25	Manuscrito propio de la entrega, compilable y sin referencias indefinidas	Manuscrito	Hecho
26	Separación entre documento vivo y documento congelado	Manuscrito	Culminar
27	Bibliografía verificada con identificador persistente	Manuscrito	Hecho
28	Preguntas de investigación formuladas y respondidas	Manuscrito	Hecho
29	Amenazas a la validez con mitigación declarada	Manuscrito	Hecho
30	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho
31	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho
32	Licencia y archivo de citación	Formalidades	Hecho
33	Actas de reunión	Formalidades	Hecho
34	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho
35	Capturas de la aplicación en release, sin composiciones	Evidencia	Hecho
36	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Hecho
37	Cobertura reportada de forma completa (líneas, instrucciones y ramas)	Pruebas	Hecho
38	Escritura consistente de los archivos de datos experimentales	Medición	Hecho
39	Sumas de verificación de los datos crudos	Medición	Hecho
40	Fijación de finales de línea para verificación reproducible	Medición	Modificar
41	Autoría distribuida y trazable sobre el manuscrito	Manuscrito	Modificar
42	Anuncio en el README de las dos rutas de resultados	Formalidades	Modificar
43	Paquete móvil firmado y publicado en release	Aplicaciones	Hecho
44	Archivos de evidencia con contenido	Evidencia	Hecho

Resumen

Estado	Entregables	Proporción
Iniciar	1	2 %
Culminar	7	16 %
Modificar	5	11 %
Hecho	31	70 %
Total	44	100 %

El trabajo pendiente se reparte así por ámbito: Arquitectura (3); Manuscrito (3); Aplicaciones (2); Datos (1); Pruebas (1); Flujo (1); Medición (1); Formalidades (1). Esa distribución sirve para repartirse el trabajo sin que dos personas se pisen.

4 El trabajo que queda, entregable por entregable

De cada entregable pendiente doy tres cosas. **La situación** es lo que encontré al revisar, con el archivo y el dato concreto, para que no tengan que adivinar de qué hablo. **Qué falta** es la acción que cierra el punto. Y el **estado** dice de dónde parte cada uno.

Entregables en Iniciar — no existe nada

5. Esquema de base de datos y migraciones

Datos · **Iniciar**

La situación. Busqué los dos motores de migraciones más extendidos en todos los archivos de proyecto, en los de configuración de los servicios y en los del clúster, y no obtuve ninguna coincidencia; tampoco existe ningún directorio de migraciones. Comprobé que la generación automática del esquema está desactivada en los tres servicios con persistencia, lo cual es correcto, pero eso solo significa que el marco ya no crea el esquema, no que exista un historial versionado que lo sustituya. El esquema real lo crean hoy tres guiones sueltos del clúster, no versionados como cambios. Abrí además la prueba con contenedor efímero y comprobé que construye el esquema con sentencias escritas dentro de la propia clase, en un arreglo que ejecuta en un bucle, de modo que ni siquiera esa prueba demuestra migraciones. No existe forma de reproducir el esquema de una versión anterior.

Qué falta

Incorporar un motor de migraciones a los servicios con persistencia, convertir los guiones del clúster en migraciones versionadas, y hacer que la prueba con contenedor aplique esas migraciones en vez de su propio código de creación.

Entregables en Culminar — el desarrollo está a medias

3. Vistas de arquitectura C4 (tres niveles)

Arquitectura · **Culminar**

La situación. Revisé qué diagramas se incluyen realmente en el manuscrito y los dos archivos de arquitectura solo insertan dos imágenes cada uno, el nivel de contexto y el de contenedores: el nivel de componentes no llega al documento. Sí existe un archivo de nivel 3, pero lo abrí y comprobé dos cosas. Primera, es un acercamiento al particionado de la tabla de tickets en la base distribuida, con sus cuatro particiones trimestrales y dos índices secundarios, no una descomposición de un contenedor en las piezas que lo forman y cómo se llaman entre sí. Segunda, de las dos únicas cajas de aplicación que nombra solo una existe en el árbol, de modo que la composición no se puede comprobar abriendo las clases que nombra. El archivo data del 26 de julio y el trabajo correctivo no lo tocó.

Qué falta

Dibujar el nivel de componentes real del servicio principal con los controladores, manejadores de aplicación, políticas de dominio y adaptadores de persistencia que existen en el árbol, versionar la fuente además de la imagen, e incluirlo en la sección de arquitectura.

4. Contrato de interfaz OpenAPI

Arquitectura · **Culminar**

La situación. Analicé el contrato recorriendo su árbol de rutas y conté 17 operaciones, de las que 4

siguen sin declarar esquema de respuesta: el cierre de sesión, el listado de tickets, el listado de notificaciones y la consulta de clasificación por ticket. Es una mejora real frente a las once anteriores, pero un cliente sigue sin poder generarse del contrato completo. La segunda parte del criterio no se cumple en absoluto: busqué en el único flujo del repositorio las cuatro herramientas habituales de validación de contratos y no hay ninguna coincidencia, de modo que no existe validador en ningún sitio. Además el documento afirma en dos líneas consecutivas que «cada operación declara su esquema de petición y respuesta», lo que hoy es falso para cuatro operaciones, y habla de «los quince endpoints REST» cuando el contrato define diecisiete.

Qué falta

Declarar el esquema de respuesta de las cuatro operaciones restantes, añadir al flujo un paso de validación que falle la construcción, y corregir en el manuscrito el número de operaciones y la afirmación de cobertura total de esquemas.

10. Capacidades de dispositivo (cámara y localización)

Aplicaciones · **Culminar**

La situación. Abrí el repositorio de datos del móvil y la función de cierre en sitio sigue enviando únicamente la actualización de estado a resuelto; el comentario inmediatamente anterior lo declara sin ambigüedad: «la foto de evidencia y la geolocalización capturadas quedan listas para enviarse en cuanto el backend exponga el campo correspondiente». Revisé la interfaz de red del móvil y solo declara tres métodos, ninguno con fotografía ni coordenadas. Comprobé también que el contrato no tiene campo de foto ni de ubicación en la creación del ticket ni en el cambio de estado, y que no hay almacenamiento en el lado del servicio. La captura de fotografía y posición sí está construida en el móvil, con sus dos botones en la pantalla de detalle: la funcionalidad está a medias y se queda dentro del teléfono.

Qué falta

Definir en el contrato la operación que recibe la evidencia fotográfica y las coordenadas, implementarla en el servicio decidiendo dónde se guarda el archivo, enviarla desde el móvil con reintento, cumplir el consentimiento de ubicación, y capturar la evidencia de los tres momentos del recorrido.

11. Integración web + móvil + backend

Aplicaciones · **Culminar**

La situación. Busqué evidencia del móvil contra el servicio y lo único existente son dos capturas del 4 de septiembre, anteriores a la revisión y sin regenerar. Abrí las dos: la del listado muestra un aviso explícito de que está sin conexión mostrando datos guardados, de modo que lo que se ve procede de la caché local y no de una respuesta del servicio; la del detalle muestra un ticket cuyo identificador busqué en todo el repositorio sin encontrar ningún registro del servicio que lo correlacione. Revisé también las pruebas: hay un solo archivo instrumentado, de la pantalla de acceso, y cinco métodos de prueba unitaria, ninguno ejercitando el recorrido contra el servicio.

Qué falta

Levantar el sistema, ejecutar una acción desde el móvil con red, y versionar una evidencia fechada que empareje la captura de la aplicación con la línea correspondiente del registro del servicio.

12. Pruebas de unidad e integración del backend

Pruebas · **Culminar**

La situación. Comprobé que la compuerta de cobertura sigue sin poder fallar, y lo demostré ejecutándola. La herramienta aparece solo en el archivo de proyecto del servicio principal, dentro de un perfil que no se activa por omisión; busqué el objetivo de comprobación, la regla y el umbral en los siete archivos de proyecto y no hay ninguna coincidencia. Revisé además el flujo, que eje-

cuta las pruebas de tres servicios sin activar ese perfil, de modo que el pipeline ni siquiera mide, pese a que el comentario del propio archivo afirma que el reporte «es usado por el pipeline». Para verificarlo ejecuté la comprobación sobre una copia: las 85 pruebas pasan, el reporte se genera, y la comprobación aborta diciendo que los parámetros de reglas faltan o no son válidos, es decir, no hay ningún umbral que subir ni bajar.

Qué falta

Sacar la herramienta del perfil inactivo, añadirle una ejecución de comprobación con umbral sobre el módulo entero, ejecutarla en el trabajo de pruebas del flujo, y verificar que subir el umbral por encima de la cifra real rompe la construcción.

16. Pipeline de integración y despliegue con publicación etiquetada

Flujo · **Culminar**

La situación. Comprobé que sí se añadió la automatización: el flujo incorpora un paso de publicación de versión con la acción correspondiente, etiqueta fija y los dos archivos del instalable, condicionado a envíos sobre la rama principal, y el README del módulo móvil lo enlaza. Ahora bien, verifiqué el resultado real y no el mensaje: el commit que la introduce sí está en la rama principal, pero al consultar las referencias del remoto las únicas etiquetas que existen son dos copias de seguridad previas; no hay etiqueta de publicación, y como toda publicación crea su etiqueta, la versión todavía no existe. Además, aunque llegara a crearse, reuniría solo el instalable y su suma, no el manuscrito que exige el criterio, y el README de la raíz no la enlaza.

Qué falta

Conseguir que el paso de publicación se ejecute de verdad, añadirle el PDF del manuscrito junto al instalable bajo una etiqueta de versión de la entrega, y enlazar esa publicación desde el README de la raíz.

26. Separación entre documento vivo y documento congelado

Manuscrito · **Culminar**

La situación. Verifiqué que existe un precedente de instantánea pero no la de esta entrega. El informe consolidado de la entrega anterior son 1.054 líneas y comprobé que no incluye por referencia ninguna sección, de modo que compila sin depender del documento vivo; pero corresponde a la entrega tres, no a la actual. Para la entrega cuatro sigue habiendo un solo árbol: el documento principal encadena veinte inclusiones de los mismos archivos que se siguen editando, y el propio README lo describe como documento único acumulado. El PDF versionado no sirve de instantánea: es la salida del documento vivo recompilada una y otra vez, la última en el propio commit de hoy, y no lleva ninguna advertencia de fecha ni de commit congelado.

Qué falta

Congelar la versión de la entrega, bien como un documento autocontenido al estilo del de la entrega anterior, bien como un PDF fechado con la advertencia del commit al que corresponde, y dejar el documento principal como fuente viva.

Entregables en Modificar — están hechos pero no son utilizables como evidencia

1. Refactorización en capas y principios SOLID

Arquitectura · **Modificar**

La situación. Hice el barrido de importaciones sobre los paquetes de dominio de los siete servicios y sigue devolviendo coincidencias: trece líneas de importación del marco en diez clases. Nueve están en el servicio principal —dos políticas de acuerdo de nivel de servicio, tres estrategias de correlación, una fábrica de tickets y la cadena de escalado con sus dos manejadores— y la décima en el servicio de telemetría. La afirmación «Cero anotaciones Spring/JPA; se prueba con JUnit puro» sigue publicada en la línea 87 de los dos archivos de arquitectura del manuscrito. Busqué

además una prueba que custodie la regla, rastreando las tres formas habituales de escribirla en todo el árbol incluidos los archivos de proyecto, y no existe ninguna.

Qué falta

Mover la anotación de las diez clases a configuraciones de la capa de infraestructura, añadir una prueba de arquitectura que falle si el dominio vuelve a importar el marco, y reescribir la afirmación del manuscrito solo cuando esa prueba pase.

22. Correspondencia exacta entre el análisis y las tablas del documento Manuscrito · [Modificar](#)

La situación. Recaliculé los tres conteos desde el dato crudo. Copié los servicios fuera del repositorio y ejecuté sus pruebas uno por uno: obtuve 121 en total, repartidas en 86, 15, 11, 8 y 1, mientras el documento sigue publicando «90/90 backend». Copié la aplicación web y ejecuté su suite: 23 archivos y 75 pruebas, frente al «78/78 web» de la misma línea. El tercer conteo, «174 pruebas automatizadas», sigue en la sección de amenazas, y sumando lo que hoy existe salen 201 unitarias más las de contrato y las de navegador, es decir del orden de 214. El único conteo que sí se reproduce es el del móvil. Ninguna de las tres cifras cuestionadas es reproducible y el documento tampoco indica el comando con el que se obtendrían.

Qué falta

Sustituir las cifras 90, 78 y 174 por las medidas, indicando junto a cada una el comando exacto que la reproduce, y obtenerlas ejecutando las suites en vez de contándolas a mano.

40. Fijación de finales de línea para verificación reproducible

Medición · [Modificar](#)

La situación. Recorrí los 772 archivos versionados leyendo cada objeto del índice y ningún archivo de texto conserva finales de Windows; los cuarenta y tres que aparecen con ese byte son todos binarios y todos están declarados como tales. Conté 708 archivos con finales de Unix aplicados de hecho y 2 con finales de Windows, que son los guiones para ese sistema declarados así a propósito. La sección del manuscrito que estaba versionada con 179 líneas en finales de Windows figura ahora con cero. El límite es doble: quedan ocho archivos sin ningún atributo y no hay una regla general de captura, de modo que una extensión nueva vuelve a entrar sin fijar; y uno de los commits del lote, cuyo mensaje anuncia la normalización del contrato, está vacío y no modifica ningún archivo.

Qué falta

Añadir la regla general de normalización para que ninguna extensión nueva entre sin fijar, y no dejar en el historial mensajes que anuncien arreglos que no ocurrieron.

41. Autoría distribuida y trazable sobre el manuscrito

Manuscrito · [Modificar](#)

La situación. Extraje el texto del PDF versionado y la tabla ya nombra a los cuatro integrantes con 75,87 %, 11,66 %, 10,42 % y 2,05 %, de modo que la omisión que tenía el entregable anterior está corregida en el archivo que un lector abre. Recaliculé esos porcentajes desde el historial mapeando los dieciséis nombres de autor a sus cuatro correos reales y en el commit que la nota cita obtuve exactamente esas cuatro cifras sobre 6.006 líneas. Re compilé el manuscrito completo y obtuve 55 páginas, las mismas del PDF versionado, sin una sola cifra distinta. El límite es que el epígrafe promete que se excluyen las dependencias y los artefactos generados y el cálculo no los excluye: 318 de esas 6.006 líneas, el 5,3 %, provienen de los archivos que produce la herramienta de bibliografía, y son justo las que suben a un integrante de un 7,96 % real a un 10,42 % publicado.

Qué falta

Excluir del cálculo los artefactos generados, como promete el epígrafe, y volver a publicar los cuatro porcentajes; o cambiar el epígrafe para que describa el método que de hecho se aplicó.

42. Anuncio en el README de las dos rutas de resultadosFormalidades · **Modificar**

La situación. Comprobé la afirmación nueva contra el código y es exacta: la constante que fija la ruta de salida del inyector de averías vale literalmente la que el índice declara. Verifiqué también el comando de reproducción contra el argumento real del otro guion, cuyo valor por omisión es el que se publica, así que la orden es ejecutable tal cual. Confirmé que existen los seis guiones que se nombran y fui más allá: ejecuté el generador del reporte de correlación sobre una copia y regeneró el análisis y la tabla del documento idénticos byte a byte a los versionados. El límite es que el índice sigue sin decir que cada carpeta lleva su manifiesto de sumas ni cómo verificarlo, y que unas líneas más arriba afirma que el manuscrito tiene 51 páginas cuando hoy tiene 55.

Qué falta

Añadir al README cómo verificar los manifiestos de sumas y corregir el recuento de páginas del manuscrito.

5 Por dónde empezar

El orden que sugiero no es el de la tabla, sino el que evita rehacer trabajo.

1. **Primero lo que está en Modificar y afecta al documento.** Son correcciones cortas y son las que un revisor detecta enseguida: una cifra que no cuadra, una afirmación que el código no sostiene, una tabla con filas contradictorias. Cerrarlas primero sube mucho la calidad percibida por poco esfuerzo.
2. **Después lo que cambia números.** Toda corrección de medición obliga a reescribir tablas y a veces conclusiones. Hacerlo antes de redactar ahorra reescribir dos veces.
3. **Luego lo que está en Culminar y en Iniciar.** Es lo que más tiempo cuesta, así que conviene repartirlo por ámbito entre los cuatro y empezar en paralelo con lo anterior.
4. **Al final la redacción, las capturas y la publicación etiquetada.** Dependen de que los números ya sean los definitivos.
5. **Y desde el primer día, la integración.** Nada de acumular ramas. Cada cosa terminada se integra a la rama principal por solicitud de incorporación revisada por un compañero distinto del autor. Dejar la integración para el último día es la forma más frecuente de perder trabajo ya hecho.

6 Del repositorio al manuscrito

Lo que ustedes tienen en el repositorio no es el manuscrito: es el material del que sale el manuscrito. Esta sección explica cómo se traduce lo uno en lo otro, porque es donde los equipos suelen perder el trabajo que ya hicieron.

La regla que ordena todo

Cada afirmación no trivial del documento tiene que poder comprobarse abriendo el repositorio. Esa es la única regla, y de ella se derivan las demás. Si el documento dice que la cobertura es del ochenta por ciento, tiene que existir el comando que produce ese ochenta por ciento. Si dice que

una campaña se ejecutó diez veces, tienen que estar los diez directorios de datos crudos. Si dice que un umbral se fijó antes de medir, la fecha del *commit* que lo fija tiene que ser anterior a la fecha de la ejecución.

Un evaluador externo no comprueba si ustedes trabajaron: comprueba si lo que afirman se sostiene. Y una afirmación que no se sostiene hace más daño que una ausencia reconocida, porque la ausencia es honesta y la afirmación falsa obliga al lector a desconfiar de todo lo demás. Los estándares de comunicación empírica en ingeniería de software insisten en este punto por encima de cualquier otro [1, 2].

De qué entregable sale cada parte del documento

Parte del manuscrito	De dónde sale
Contexto y problema	Los registros de decisión de arquitectura y el análisis del dominio. El problema tiene que estar cuantificado, no enunciado.
Arquitectura del sistema	Las vistas C4 en sus tres niveles, con la fuente versionada, y el contrato de interfaz. Cada tecnología que el diagrama nombra debe aparecer en el archivo de dependencias del componente al que se atribuye.
Datos y consistencia	El esquema, las migraciones y la evidencia de fragmentación, replicación y tolerancia a fallos. La posición en el teorema CAP se argumenta por agregado del dominio, no para todo el sistema.
Procesamiento paralelo	El trabajo distribuido con su curva de aceleramiento y el ajuste de la fracción no paralelizable. Si el resultado es una desaceleración, se reporta como tal y se explica.
Diseño experimental	El protocolo, los umbrales prerregistrados y el instrumental de medición. Los umbrales se declaran antes de medir y la fecha del <i>commit</i> lo demuestra [1].
Resultados y análisis	Los datos crudos, el análisis estadístico y las figuras. En comparaciones de rendimiento se usan pruebas no paramétricas y se reporta el tamaño del efecto junto al valor de significación, nunca el valor de significación solo [3, 4].
Evaluación de calidad	Las características medidas con su valor, su intervalo y su umbral, según el modelo de calidad de producto vigente [5]. Una característica que no cumple se consigna como no cumplida.
Amenazas a la validez	Todo lo que quedó en Modificar o Culminar y no se pudo cerrar, dicho con sus palabras y con su alcance real.
Ética	La reflexión referida al código de ética profesional [6], con el tratamiento de los datos personales que el sistema maneja.

Cinco errores que un revisor detecta en la primera lectura

1. **La medición que elige su propio alcance favorable.** Publicar una cobertura calculada sobre los archivos que las pruebas ya tocan, o una compuerta aplicada a las clases que ya pasan. La cifra sube y deja de describir el proyecto. Si hace falta excluir algo, se excluye y se justifica cada exclusión una por una.
2. **El número que no se reproduce.** Una cifra escrita a mano en una tabla se desactualiza en cuanto el código cambia. Toda tabla de resultados debería salir de un guion que lea el dato crudo y emita el bloque del documento.
3. **La prueba que no puede fallar.** Una prueba que pasa igual con el sistema apagado, con la aserción invertida o con la línea que debía verificar borrada, no prueba nada. Compruébenlo ustedes antes de publicarla: rompan algo y miren si se pone en rojo.
4. **El instrumento que altera lo que mide.** La instrumentación tiene coste, y ese coste entra

en las cifras que se publican [7]. Conviene declarar en qué condiciones se midió y con qué instrumentación activa.

5. **El documento que va por detrás del repositorio.** El PDF versionado se desfasa del archivo fuente en cuanto alguien corrige el fuente y no recompila. Un trabajo del flujo automático que compile y compare cierra ese agujero para siempre.

Sobre la medición de un sistema distribuido

Medir un sistema de microservicios tiene dificultades propias que conviene nombrar en el documento en lugar de esquivarlas. La carga que se aplica debe ejercitar el camino crítico y no solo las lecturas públicas, porque son las operaciones que escriben las que entran en conflicto. La escalabilidad se mide con un método declarado y repetible, no con una corrida suelta [8]. Y las dificultades que los propios profesionales reportan al probar y monitorizar estos sistemas —la dependencia del entorno, el coste de levantar todo para una prueba, la fragilidad de las pruebas de extremo a extremo— son un marco legítimo para discutir las limitaciones del trabajo [9, 10].

7 Criterios de piso

Estos criterios no puntúan: se comprueban antes de calificar, y el incumplimiento de cualquiera de ellos deja la calificación en **CERO**.

Criterios de piso

1. **El PDF en el SGA con la dirección del repositorio.** Cada estudiante sube al SGA un documento PDF con carátula de identificación que muestre, en una sola línea y de forma claramente legible, la dirección del repositorio donde está el trabajo. Si esa dirección está mal escrita, el repositorio no es accesible, o no se cumple esta directriz, la calificación es CERO.
2. **El PDF se regenera desde el LaTeX.** Clono el repositorio, compilo el archivo principal y tiene que salir el mismo PDF. Las instrucciones de compilación —compilador, orden de comandos, archivo principal y paquetes necesarios— deben estar escritas en el README. Si el PDF no se genera de esa manera, la calificación es CERO.
3. **Todo integrado en la rama principal.** La entrega es un *commit* etiquetado de la rama principal. Si al clonar esa rama falta trabajo que ustedes consideran entregado porque vive en otra rama, la calificación es CERO.
4. **Cada uno responde por lo suyo.** En la defensa, cada integrante tiene que explicar el código y las mediciones que se le atribuyen: por qué están escritas así, qué pasa si se cambia tal cosa, de dónde salió tal número. El contenido que su autor declarado no sepa explicar se retira de la evaluación; si eso ocurre de forma sistemática en un integrante, su calificación es CERO. Esto aplica igual sea el código de un compañero, de un tutorial o de una herramienta automática: lo que se entrega, se entiende.

8 Lista de comprobación antes de entregar

1. Clonar el repositorio en un directorio vacío y compilar el documento siguiendo únicamente lo que dice el README. Si no sale el mismo PDF, todavía no está listo.
2. Ejecutar todas las suites de pruebas sobre ese clon limpio y comprobar que pasan.
3. Ejecutar todas las verificaciones de sumas y comprobar que pasan enteras.

4. Abrir el PDF y comprobar que no hay ninguna referencia ni cita sin resolver.
5. Recorrer las cifras del documento y comprobar, una por una, que cada una se reproduce con el comando que el propio documento indica.
6. Comprobar que ninguna rama va por delante de la principal.
7. Comprobar que el flujo automático está en verde en el *commit* que se entrega.
8. Crear la etiqueta de versión y la publicación con el documento y el instalable.
9. Subir al SGA el PDF con carátula y la dirección del repositorio en una sola línea.

Referencias

- [1] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.
- [2] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [3] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [4] A. Vargha and H. D. Delaney, “A critique and improvement of the CL common language effect size statistics of McGraw and Wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [5] International Organization for Standardization, “ISO/IEC 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model,” standard, ISO, Geneva, Switzerland, 2023.
- [6] Association for Computing Machinery, “ACM code of ethics and professional conduct.” <https://www.acm.org/code-of-ethics>, 2018.
- [7] Y. Hammad, A. Al-Said Ahmad, and P. Andras, “An empirical study on the performance overhead of code instrumentation in containerised microservices,” *Journal of Systems and Software*, vol. 230, p. 112573, 2025.
- [8] S. Henning and W. Hasselbring, “A configurable method for benchmarking scalability of cloud-native applications,” *Empirical Software Engineering*, vol. 27, no. 6, p. 143, 2022.
- [9] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, “Design, monitoring, and testing of microservices systems: The practitioners’ perspective,” *Journal of Systems and Software*, vol. 182, p. 111061, 2021.
- [10] M. Hui, L. Wang, H. Li, R. Yang, Y. Song, H. Zhuang, D. Cui, and Q. Li, “Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,” *Journal of Systems and Software*, vol. 220, p. 112232, 2025.

Anexo. Rúbrica de calificación

La tabla reproduce los entregables de la sección 3 con una columna adicional: el porcentaje de la calificación de la entrega que corresponde a cada uno. Los 31 entregables en **Hecho** siguen en Hecho y no se califican en esta entrega; los 13 restantes reparten el 100 % en proporción a la profundidad, la complejidad, la amplitud y el tiempo de trabajo que exige llevarlos a Hecho, con la calidad descrita en la sección 4, desde el estado en que se encontraban el domingo 6 de septiembre de 2026. Se califica el estado del repositorio en su último *commit* hasta el **viernes 18 de septiembre de 2026 a las 23:55**.

#	Entregable	Ámbito	Estado	% calificación
1	Refactorización en capas y principios SOLID	Arquitectura	Modificar	9,1 %
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho	—
3	Vistas de arquitectura C4 (tres niveles)	Arquitectura	Culminar	6,8 %
4	Contrato de interfaz OpenAPI	Arquitectura	Culminar	9,1 %
5	Esquema de base de datos y migraciones	Datos	Iniciar	9,8 %
6	Persistencia distribuida operando	Datos	Hecho	—
7	Aplicación web funcional (construye y ejecuta)	Aplicaciones	Hecho	—
8	Calidad de la aplicación web (pruebas de unidad)	Pruebas	Hecho	—
9	Aplicación móvil funcional	Aplicaciones	Hecho	—
10	Capacidades de dispositivo (cámara y localización)	Aplicaciones	Culminar	11,5 %
11	Integración web + móvil + backend	Aplicaciones	Culminar	7,6 %
12	Pruebas de unidad e integración del backend	Pruebas	Culminar	9,1 %
13	Pruebas de contrato	Pruebas	Hecho	—
14	Pruebas de extremo a extremo	Pruebas	Hecho	—
15	Pruebas de carga	Pruebas	Hecho	—
16	Pipeline de integración y despliegue con publicación etiquetada	Flujo	Culminar	8,3 %
17	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Hecho	—
18	Procesamiento paralelo	Medición	Hecho	—
19	Módulo de correlación implementado y probado	Medición	Hecho	—
20	Protocolo experimental y campaña ejecutada según diseño	Medición	Hecho	—
21	Análisis estadístico con prueba no paramétrica y tamaño de efecto	Medición	Hecho	—
22	Correspondencia exacta entre el análisis y las tablas del documento	Manuscrito	Modificar	8,3 %
23	Datos crudos preservados	Medición	Hecho	—
24	Evaluación de calidad con las características medidas	Medición	Hecho	—
25	Manuscrito propio de la entrega, compilable y sin referencias indefinidas	Manuscrito	Hecho	—
26	Separación entre documento vivo y documento congelado	Manuscrito	Culminar	5,3 %
27	Bibliografía verificada con identificador persistente	Manuscrito	Hecho	—
28	Preguntas de investigación formuladas y respondidas	Manuscrito	Hecho	—
29	Amenazas a la validez con mitigación declarada	Manuscrito	Hecho	—
30	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho	—
31	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho	—

#	Entregable	Ámbito	Estado	% calificación
32	Licencia y archivo de citación	Formalidades	Hecho	—
33	Actas de reunión	Formalidades	Hecho	—
34	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho	—
35	Capturas de la aplicación en release, sin composiciones	Evidencia	Hecho	—
36	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Hecho	—
37	Cobertura reportada de forma completa (líneas, instrucciones y ramas)	Pruebas	Hecho	—
38	Escritura consistente de los archivos de datos experimentales	Medición	Hecho	—
39	Sumas de verificación de los datos crudos	Medición	Hecho	—
40	Fijación de finales de línea para verificación reproducible	Medición	Modificar	4,5 %
41	Autoría distribuida y trazable sobre el manuscrito	Manuscrito	Modificar	6,8 %
42	Anuncio en el README de las dos rutas de resultados	Formalidades	Modificar	3,8 %
43	Paquete móvil firmado y publicado en release	Aplicaciones	Hecho	—
44	Archivos de evidencia con contenido	Evidencia	Hecho	—
Total (13 entregables calificados)				100,0 %

Guía para culminar el Proyecto Fin de Curso

Equipo AGLS

Aplicaciones Distribuidas (ISR-701) · Séptimo semestre · Carrera de Ingeniería de Software
Facultad de Ciencias de la Computación y Diseño Digital · UTEQ · PPA 2026–2027

El proyecto

TiendaTech — Comercio electrónico distribuido con asistente de armado

Repositorio: <https://github.com/JoseLozanoMorales/TiendaTech>

Estado revisado sobre la rama main, *commit* 822399a del 13 de septiembre de 2026, con 309 *commits* de historia.

Integrantes: Jhinson Aucatoma, Jeremy Gaibor, Andy Sánchez, José Lozano.

1 Para qué sirve esta guía

Esta guía responde a una sola pregunta: qué les falta para cerrar el Proyecto Fin de Curso con un manuscrito que se sostenga. Para responderla revisé el repositorio entero sobre el último *commit*, entregable por entregable, sin límite de fecha. No me quedé en comprobar que los archivos estuvieran: ejecuté las suites de pruebas, medí yo mismo la cobertura sin los filtros del proyecto, recalculé desde los datos crudos las cifras que el documento publica, ejecuté las verificaciones de sumas contando cuántas pasan y cuántas fallan, y comprobé en cada caso que las pruebas pudieran fallar rompiendo una aserción o el dato.

El resultado es la tabla de la sección 3: los **47 entregables** que el proyecto debía desarrollar, cada uno con su estado. De ellos, 30 están en Hecho y 17 necesitan trabajo. Ese es el alcance real de lo que queda.

Lectura del estado del proyecto

Este proyecto tiene el mejor sistema de trazabilidad del curso: el validador de evidencias funciona de verdad, lo comprobé inyectándole tres defectos distintos, y corre en el flujo antes de compilar. El trabajo correctivo fue además sustancial: la cobertura web pasó de tres vistas a doce, el manifiesto de datos pasó de cubrir un archivo a cubrir los sesenta, el análisis de complejidad se convirtió en compuerta real y el instalable dejó de estar firmado con la clave de depuración. Lo que impide cerrar el PFC es lo que se mide y lo que se cuenta: el contrato describe 28 de 119 operaciones, las pruebas de extremo a extremo interceptan todo el tráfico y responden con datos fijos, no hay verificación de contratos desde el proveedor, la carga solo ejercita cuatro lecturas públicas, y el manuscrito sigue publicando las cifras de cobertura anteriores al trabajo que ustedes mismos hicieron.

2 Los cuatro estados

Estado	Qué significa exactamente
Hecho	El entregable está terminado, es correcto al cien por cien y está listo para usarse en la redacción del manuscrito. Se puede citar tal como está: la cifra que publica se reproduce, el archivo que nombra existe, la prueba que declara puede fallar.
Modificar	El entregable está terminado, pero no está correctamente desarrollado. No está listo para la redacción: usarlo tal como está introduciría en el manuscrito un error o una afirmación que el repositorio no sostiene.
Culminar	El desarrollo está iniciado y hay que continuarlo. La pieza existe y funciona en parte, pero le falta lo que la haría utilizable como evidencia.
Iniciar	El entregable no está presente en el proyecto. Hay que construirlo desde cero.

La diferencia entre Modificar y Culminar, que es la que más se confunde

Un entregable en **Modificar** ya está construido y el trabajo que queda es de corrección: cambiar un número que no cuadra, quitar una afirmación que el código no sostiene, retirar un filtro que infla una medición. Suele costar poco tiempo y suele ser lo que más daño hace si se deja, porque un revisor externo lo detecta abriendo el repositorio y pierde la confianza en todo lo demás.

Un entregable en **Culminar** tiene el trabajo a medias y lo que queda es construcción: ejecutar una campaña que no se ejecutó, escribir el verificador que falta, capturar la evidencia que nadie capturó. Cuesta más tiempo, pero mientras está a medias no engaña a nadie.

Por eso el orden de ataque que propongo más adelante empieza por lo que está en Modificar y no por lo que parece más grande.

3 Todos los entregables, con su estado

La tabla recoge todo lo que el proyecto debía desarrollar, incluido lo que ya está resuelto. La columna de estado es la que manda: lo que aparece en Hecho no hay que volver a tocarlo, y puede citarse en el manuscrito tal como está.

#	Entregable	Ámbito	Estado
1	Refactorización en capas y principios SOLID	Arquitectura	Hecho
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho
3	Vistas de arquitectura C4 (tres niveles) versionadas como fuente	Arquitectura	Hecho
4	Contrato de interfaz OpenAPI	Arquitectura	Culminar
5	Esquema de base de datos y migraciones	Datos	Culminar
6	Persistencia distribuida operando con fragmentación y réplica	Datos	Hecho
7	Aplicación web funcional (construye y ejecuta)	Aplicaciones	Hecho
8	Aplicación móvil funcional	Aplicaciones	Hecho
9	Capacidades de dispositivo (cámara y notificaciones)	Aplicaciones	Hecho
10	Integración web + móvil + backend	Aplicaciones	Hecho
11	Pruebas de unidad e integración del backend	Pruebas	Hecho
12	Pruebas de contrato	Pruebas	Culminar
13	Pruebas de extremo a extremo	Pruebas	Modificar

#	Entregable	Ámbito	Estado
14	Pruebas de carga	Pruebas	Culminar
15	Cobertura y complejidad reportadas con valores verificables	Pruebas	Modificar
16	Pipeline de integración con compuerta de calidad efectiva	Flujo	Hecho
17	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Modificar
18	Procesamiento paralelo	Medición	Hecho
19	Protocolo experimental e instrumental de medición	Medición	Hecho
20	Campaña ejecutada y totales reproducibles desde el dato crudo	Medición	Hecho
21	Análisis estadístico con tamaño de efecto e intervalos	Medición	Hecho
22	Datos crudos preservados	Medición	Modificar
23	Evaluación de calidad con las características medidas	Medición	Hecho
24	Manuscrito compilable, sin referencias indefinidas ni figuras huérfanas	Manuscrito	Hecho
25	Verificación del hash del artefacto documental	Manuscrito	Hecho
26	Bibliografía verificada con identificador persistente	Manuscrito	Modificar
27	Preguntas de investigación formuladas y respondidas	Manuscrito	Hecho
28	Amenazas a la validez tratadas por tipo y con mitigación	Manuscrito	Hecho
29	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho
30	Corrección explícita de afirmaciones previas no sostenidas	Manuscrito	Hecho
31	Integración de las ramas de trabajo a la principal	Flujo	Culminar
32	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho
33	Licencia y archivo de citación	Formalidades	Hecho
34	Actas de reunión	Formalidades	Culminar
35	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho
36	Capturas de la aplicación en release	Evidencia	Hecho
37	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Iniciar
38	Pruebas de unidad de la aplicación web	Pruebas	Modificar
39	Análisis estático de la aplicación web	Pruebas	Hecho
40	Publicación de imágenes en el registro prescrito	Flujo	Hecho
41	Provisión de tableros sin duplicación	Observabilidad	Hecho
42	Sumas de verificación de los datos crudos	Medición	Modificar
43	Todo archivo de evidencia citado existe en el árbol	Manuscrito	Hecho
44	Paquete móvil firmado y publicado en release	Aplicaciones	Modificar
45	Raíz del repositorio sin carpetas huérfanas	Formalidades	Hecho
46	Fijación de finales de línea para verificación reproducible	Medición	Culminar
47	Publicación etiquetada de la entrega	Flujo	Culminar

Resumen

Estado	Entregables	Proporción
Iniciar	1	2 %
Culminar	8	17 %
Modificar	8	17 %
Hecho	30	64 %
Total	47	100 %

El trabajo pendiente se reparte así por ámbito: Pruebas (5); Medición (3); Flujo (2); Formalidades (2); Arquitectura (1); Datos (1); Observabilidad (1); Manuscrito (1); Aplicaciones (1). Esa distribución sirve para repartirse el trabajo sin que dos personas se pisen.

4 El trabajo que queda, entregable por entregable

De cada entregable pendiente doy tres cosas. **La situación** es lo que encontré al revisar, con el archivo y el dato concreto, para que no tengan que adivinar de qué hablo. **Qué falta** es la acción que cierra el punto. Y el **estado** dice de dónde parte cada uno.

Entregables en Iniciar — no existe nada

37. Higiene del repositorio (sin artefactos de compilación versionados) Formalidades · Iniciar

La situación. Medí el peso real de lo versionado sobre un clon limpio y hoy el árbol rastrea 378,3 MB en 1.494 archivos, es decir más que los 343 MB anteriores. No se ha retirado nada: siguen el video de 90,2 MB, el instalable de depuración de 39,1 MB, el agente de instrumentación de 22,3 MB y 360 archivos de base de datos que suman 156,5 MB. Además el trabajo correctivo empeoró el punto, porque dos commits del 12 y 13 de septiembre añadieron un segundo instalable de 34,2 MB. Verifiqué el lado positivo para no atribuir un defecto que no existe: las reglas de exclusión sí cubren los directorios de compilación y de dependencias, y un commit dejó de versionar la configuración local del entorno de desarrollo. Pero ninguna de esas acciones toca los artefactos pesados, y como todo sigue en el historial, adelgazarlo más adelante exigiría reescribirlo.

Qué falta

Retirar del árbol el video, los dos instalables, el agente de instrumentación y los 360 archivos de base de datos, publicarlos como adjuntos de la versión etiquetada, y dejar en el README el enlace desde el que cada uno es alcanzable.

Entregables en Culminar — el desarrollo está a medias

4. Contrato de interfaz OpenAPI

Arquitectura · Culminar

La situación. Verifiqué que el directorio del contrato no ha sido tocado desde el 4 de septiembre, es decir que ninguno de los treinta y dos commits correctivos trabajó sobre él. Conté las operaciones realmente documentadas resolviendo cada referencia contra su archivo de servicio y suman 28. Contra eso conté las rutas expuestas en el código: 113 anotaciones de ruta en clases de control, más 3 de nivel de método y 3 rutas del servicio de inteligencia artificial, lo que da 119 operaciones frente a 28 descritas: quedan 91 sin documentar. Busqué además un mecanismo de regeneración y una compuerta automática y no existe ninguno, de modo que hoy se puede añadir una ruta sin documentarla y el flujo sigue en verde. Lo que sí está resuelto es la estructura: los siete contratos por servicio existen y el archivo consolidado los agrega por referencia sin duplicar definiciones.

Qué falta

Generar el contrato desde el propio código con la herramienta del marco, declarar esquema de petición y de respuesta de las 119 operaciones, y añadir al flujo un trabajo que compare el número de rutas del código con el del contrato y falle si no coinciden.

5. Esquema de base de datos y migracionesDatos · **Culminar**

La situación. Busqué un motor de migraciones en todo el árbol y no existe: ninguna aparición en los seis archivos de proyecto, ni en los de configuración, ni en los flujos. Verifiqué cómo se crea realmente el esquema y es por aplicación de un archivo monolítico de 504 líneas con 33 sentencias de creación de tabla, montado por los dos archivos de composición. Comprobé también que la generación automática desde las clases está desactivada, lo cual corrige la parte más grave, pero no la sustituye por un historial versionado. Existe un directorio con tres migraciones numeradas al estilo del motor habitual y con contenido idempotente, pero busqué su invocación en todo el repositorio y ningún guion, ninguna composición y ningún flujo las aplica: son código muerto, faltan las tres primeras versiones, y un clon limpio nunca las ejecuta.

Qué falta

Incorporar el motor de migraciones a los seis servicios, convertir el archivo monolítico en la migración base seguida de las tres ya escritas, retirar su carga del archivo de composición, y añadir al flujo un trabajo que levante la base desde cero solo con migraciones.

12. Pruebas de contratoPruebas · **Culminar**

La situación. Inspeccioné el directorio de contratos y sigue habiendo únicamente lado consumidor: dos pruebas que generan los dos contratos, cada uno con una sola interacción y su estado de proveedor declarado, lo que significa que la materia prima que necesitaría un verificador ya está escrita. Busqué en todo el repositorio los cuatro términos con los que se escribe una verificación de proveedor, sobre Java, JavaScript y los archivos de configuración, y no hay ni una sola aparición. El trabajo del flujo solo regenera el contrato del consumidor y lo archiva como artefacto. El propio índice del directorio lo admite al escribir que los estados «documentan las precondiciones que deben implementar los verificadores», verificadores que no existen. La consecuencia práctica la comprobé: si hoy se cambia la forma de una respuesta, nada en el flujo se rompe, que es justo lo que el contrato debía detectar.

Qué falta

Añadir la verificación del proveedor en los dos servicios con los contratos versionados y sus estados, engancharla al flujo, y demostrar con un cambio deliberado de la forma de una respuesta que la verificación falla.

14. Pruebas de cargaPruebas · **Culminar**

La situación. Leí el guion de carga y sigue idéntico: una sola clase con cuatro tareas, todas de lectura sin autenticación, sobre catálogo, categorías, marcas y provincias. No hay ninguna petición de escritura, ni cabecera de autorización, ni referencia al carrito, al pago ni al asistente de armado. Recaliculé las cifras desde el dato crudo y la campaña publicada son 2.624 peticiones, 0 fallos, mediana de 8 ms y percentil 95 de 17 ms. Sí hubo trabajo correctivo real en dos de los tres requisitos: los cuatro archivos crudos están versionados con su manifiesto, ejecuté la verificación sobre un clon limpio y las cuatro sumas pasan, y el trabajo de integridad del flujo las ejecuta; además la tasa de error viene desglosada por operación. Lo que no se tocó es lo esencial: la campaña mide el catálogo público y no el sistema, de modo que el cero por ciento de error no dice nada sobre las operaciones que escriben.

Qué falta

Añadir al guion tareas autenticadas del camino crítico —sesión, carrito, pedido, pago y asistente— con renovación del testigo, repetir la campaña con varios niveles de concurrencia, versionar los nuevos crudos y regenerar el manifiesto.

31. Integración de las ramas de trabajo a la principalFlujo · **Culminar**

La situación. Recorrí las siete ramas remotas y calculé para cada una cuántos commits lleva por delante y por detrás de la principal. Cinco están hoy en cero por delante, es decir que su trabajo quedó integrado, y eso se ve también en el historial reciente con cuatro fusiones. Una rama sigue exactamente igual que antes: un commit por delante y 306 por detrás. Ese commit es del 9 de agosto, toca nueve archivos con 824 líneas añadidas, entre ellos tres páginas de la interfaz y un modelo de órdenes de compra. Comprobé sobre el commit de hoy que ninguno de esos archivos existe en la rama principal bajo ningún nombre, así que no es trabajo duplicado: es contenido que quien clone la rama principal sigue sin tener. La rama tampoco está documentada ni mencionada en el índice.

Qué falta

Decidir sobre ese commit integrándolo por solicitud de incorporación si su contenido aporta, o eliminando la rama y dejando constancia de la decisión, de modo que ninguna rama quede por delante al cierre.

34. Actas de reuniónFormalidades · **Culminar**

La situación. Listé el directorio de actas y sigue habiendo un solo archivo, de 4.317 bytes, incorporado el 4 de septiembre; ninguno de los treinta y dos commits correctivos añadió actas. Busqué en todo el árbol cualquier otro documento cuyo nombre contuviera la palabra y no aparece nada fuera de ese directorio. Revisé también la bitácora del proyecto por si cumpliera la función y no la cumple: tiene 46 líneas, documenta un único corte técnico del 31 de agosto y es un registro de estado, no un acta con decisiones y responsables. El historial sigue mostrando coordinación real y sostenida entre los cuatro integrantes durante todo septiembre, pero no hay registro de en qué reunión se decidió cada cosa ni quién quedó a cargo. La única acta que existe está correctamente fechada y registrada el mismo día de la sesión, de modo que sirve de plantilla.

Qué falta

Redactar un acta por cada reunión del periodo tomando como plantilla la existente, y registrarlas con fecha posterior a la reunión que documentan.

46. Fijación de finales de línea para verificación reproducibleMedición · **Culminar**

La situación. Reproduce el comando del flujo sobre el árbol entregado y sale en verde, pero al examinar qué cubre encontré que la ampliación anunciada es inerte. El patrón de archivos separados por tabulación no selecciona ningún archivo del repositorio. El patrón de los cuatro archivos de evidencia selecciona archivos codificados en UTF-16 que ya contienen finales de Windows en el propio objeto versionado, de modo que incumplen desde el primer día el atributo que se les asigna; y el comando los descarta por binarios y ni siquiera los mira, lo que comprobé convirtiendo uno a finales de Windows completos y viendo que el trabajo seguía en verde, mientras que el mismo experimento sobre un archivo tabular real sí se detecta. El alcance efectivo del control sigue siendo el mismo que antes, y no existe ninguna regla global de normalización, así que el código fuente sigue sin fijar.

Qué falta

Añadir la regla general de normalización, retirar del control los patrones que no seleccionan nada, convertir los cuatro archivos de evidencia a codificación de un byte con finales de Unix, y comprobar que el guardián los inspecciona de verdad.

47. Publicación etiquetada de la entregaFlujo · **Culminar**

La situación. Listé las etiquetas locales y remotas y ahora hay 36, frente a la ausencia total anterior: además de la que marca el punto de partida congelado, hay 31 de compilaciones de depuración y 4 de entrega. Leí el trabajo de publicación del flujo y es real y automático: verifica la suma del paquete, valida el certificado contra una huella fijada y crea la publicación con el instalable, su suma, la salida del verificador de firma y un archivo de procedencia; la evidencia registra una publicación concreta con su ejecución en verde. Lo que falta es que eso sea la versión de la entrega: el propio comando la crea como prelanzamiento, la etiqueta no es una versión sino el identificador numérico de la ejecución, y sus adjuntos son solo el instalable, sin el manuscrito. El índice no enlaza ninguna publicación, y el archivo de citación declara una versión para la que no existe etiqueta.

Qué falta

Crear la etiqueta de versión de la entrega, publicarla como versión definitiva reuniendo el manuscrito y los artefactos junto al instalable, alinearla con la versión del archivo de citación y enlazarla desde el README.

Entregables en Modificar — están hechos pero no son utilizables como evidencia**13. Pruebas de extremo a extremo**Pruebas · **Modificar**

La situación. Abrí los tres archivos de pruebas de navegador y siguen exactamente como estaban; ninguno de los treinta y dos commits correctivos los tocó. El interceptor registra una ruta comodín sobre todas las llamadas al servidor y responde toda petición a partir de una constante escrita en el propio archivo; busqué las tres formas de dejar pasar una petición al servidor real y no aparece ninguna, de modo que está demostrado que ninguna llamada sale jamás. Revisé la configuración y solo arranca el frontal, sin base de datos ni microservicios, y revisé el trabajo del flujo, cuyos pasos son únicamente instalar, descargar el navegador y ejecutar: la suite pasa en el flujo con el sistema completo ausente, que es la definición de que no puede fallar por una ruptura del servidor. De las tres pruebas, dos usan el interceptor y la tercera solo verifica un redireccionamiento del lado del cliente.

Qué falta

Levantar el sistema real en el trabajo del flujo y esperar a que responda, eliminar el interceptor global dejando a lo sumo simulaciones puntuales de fallo, y demostrar con una corrida con el sistema apagado que la suite falla.

15. Cobertura y complejidad reportadas con valores verificablesPruebas · **Modificar**

La situación. Recaliculé la cobertura desde el dato crudo y no desde el documento. Los seis archivos de proyecto mantienen una restricción que limita la medición a un paquete por servicio, y la configuración web restringe la cobertura a una lista explícita de doce vistas. Copié los seis servicios fuera, quité la restricción y ejecuté: 11,72 %, 22,66 %, 32,66 %, 41,25 % y 41,14 %, lo que da un agregado real de 32,97 % frente al 82,31 % publicado. Encontré además un defecto que antes no estaba a la vista: los seis informes versionados son del 28 de agosto y ya no reproducen el árbol de hoy, porque al reejecutar con los filtros originales obtuve cifras distintas en cuatro de los cinco

servicios; como el verificador de umbral lee esos informes congelados, el informe certifica un árbol que ya no existe. En el lado web hoy corren 62 pruebas con 56,08 % de sentencias, mientras el índice de evidencias sigue afirmando 19 pruebas y 30,78 %.

Qué falta

Retirar la restricción de los seis archivos de proyecto y la lista de doce vistas, regenerar todos los informes desde una corrida actual, publicar la cifra sobre el árbol completo y justificar cada exclusión que se decida conservar.

17. Observabilidad: métricas, trazas, registros y tableros

Observabilidad · **Modificar**

La situación. Abrí la configuración del recolector y sigue teniendo siete objetivos, los seis servicios Java y el de inteligencia artificial; el servicio de puerta de entrada existe en el archivo de composición y no figura en ninguna lista de objetivos, así que el recolector sigue sin verlo. Extraje las consultas del tablero y abrí las dos capturas publicadas: en la corrida conjunta el panel de errores de cliente muestra literalmente «No data», y en la otra ese panel solo dibuja una serie plana en cero mientras el de errores de servidor solo dibuja un servicio. La propia documentación del equipo reconoce la causa al escribir que esos códigos los responde la puerta de entrada antes de reenviar y que la puerta «no está entre los objetivos». El tablero está bien construido y los paneles de peticiones, latencia y conexiones sí tienen series, pero publicar esa captura como evidencia de observabilidad afirma una cobertura de medición que la propia imagen desmiente.

Qué falta

Añadir la puerta de entrada a los objetivos del recolector, repetir la corrida de carga y sustituir las capturas por unas en las que los cinco paneles muestren series con datos.

22. Datos crudos preservados

Medición · **Modificar**

La situación. Localicé el archivo de metodología del análisis estadístico y verifiqué primero que los ocho archivos a los que apunta sí existen hoy con esos nombres exactos, lo cual corrige la mitad del defecto anterior. Después cloné el repositorio limpio y recalculé las ocho sumas: fallan las ocho, ninguna pasa. Investigué la causa comparando variantes y las ocho coinciden únicamente si se convierte el contenido a finales de Windows, es decir que fueron calculadas sobre un clon de ese sistema, mientras el archivo de atributos declara finales de Unix para esos archivos y por tanto en cualquier clon limpio del flujo la verificación no puede pasar nunca. Peor aún, el propio repositorio se contradice: el manifiesto general registra para tres de esos archivos valores distintos de los que declara el archivo de metodología, y los del manifiesto son los que yo obtengo del archivo real.

Qué falta

Regenerar las ocho sumas sobre el contenido normalizado tal como se publica, alinearlas con el manifiesto general, y añadir su comprobación al trabajo de integridad del flujo.

26. Bibliografía verificada con identificador persistente

Manuscrito · **Modificar**

La situación. Comprobé que la bibliografía no ha sido modificada desde el 4 de septiembre. Extraje los 60 identificadores de objeto digital de las 70 entradas y los resolví uno a uno contra el registro público comparando el título devuelto con el de cada entrada. Cincuenta y ocho resuelven a la obra correcta y los dos erróneos siguen donde estaban: una entrada sobre el uso de flujos de integración continua en repositorios lleva un identificador que resuelve a un banco de pruebas de defectos en COBOL, un trabajo distinto de otros autores; y una entrada sobre la percepción de los desarrolladores respecto de esos flujos lleva un identificador que resuelve a un artículo sobre barreras de inclusión social en comunidades de ingeniería de software, vecino en las mismas actas. Descarté un tercer caso sospechoso, que resulta correcto y solo tiene el título abreviado. No existe

ningún guion que valide identificadores, por lo que nada impediría que vuelva a ocurrir.

Qué falta

Sustituir los dos identificadores por los que corresponden a esos artículos y verificar la resolución de los sesenta antes de compilar el manuscrito.

38. Pruebas de unidad de la aplicación web

Pruebas · **Modificar**

La situación. Ejecuté la suite completa sobre una copia y obtuve 62 pruebas en 14 archivos, frente a las 19 anteriores, y comprobé que la configuración ya instrumenta las doce vistas y no tres. Para medir sin sus restricciones escribí mi propia configuración sobre el árbol completo y recalculé: 56,22 % de sentencias, 45,10 % de ramas y 44,47 % de funciones, cifras prácticamente iguales a las que publican con su filtro, de modo que el defecto de la medición acotada está realmente corregido. Comprobé además que las pruebas nuevas pueden fallar: inyecté diez mutaciones en el código de producción y ocho dieron rojo; las dos supervivientes corresponden a comportamientos que las pruebas no afirman, no a aserciones huecas. El límite es que el manuscrito no se actualizó: sigue publicando 19 pruebas y 53,39 % de líneas sobre tres vistas cuando el trabajo real es muy superior.

Qué falta

Llevar al manuscrito las cifras de hoy —62 pruebas y la cobertura medida sobre las doce vistas— y retirar el resumen antiguo que el texto enlaza.

42. Sumas de verificación de los datos crudos

Medición · **Modificar**

La situación. Ejecuté el verificador global y comprobó 60 sumas, exactamente los 60 archivos tabulares del índice, frente al único archivo que cubría antes; el verificador por campaña comprobó otras 16, y las once pruebas negativas de ambos pasaron. Cloné el repositorio a un árbol desechable y provoqué cuatro alteraciones —un byte añadido, una línea cambiada, un archivo nuevo sin suma y una línea borrada del manifiesto— y las cuatro fueron detectadas con código de error. El flujo de integridad corre en envío y en solicitud de incorporación sin tolerancia al error y sin regenerar nunca el manifiesto. El límite es el alcance: el inventario es deliberadamente tabular y deja sin suma nueve archivos estructurados de las propias carpetas de campaña, entre ellos los que contienen los veredictos del experimento.

Qué falta

Ampliar el inventario a los archivos estructurados que contienen los veredictos, de modo que la evidencia que sostiene las conclusiones quede también cubierta.

44. Paquete móvil firmado y publicado en release

Aplicaciones · **Modificar**

La situación. Extraje yo mismo el bloque de firma del paquete versionado y el titular del certificado es el nombre propio de un integrante, autofirmado, con clave de 3072 bits y validez hasta 2054, y su huella es idéntica a la del certificado público versionado; para contrastar extraje el del paquete de depuración que sigue en la misma carpeta, que sí lleva el titular genérico, así que el defecto anterior está resuelto. El archivo de construcción declara ya una configuración de firma alimentada por cuatro variables y registra una tarea que aborta si falta alguna, unida por dependencia a las tres tareas de empaquetado. El límite es que el binario entregado no es reproducible desde el repositorio: el archivo de verificación registra un resumen del guion de construcción que no coincide con el del archivo versionado en ningún commit, y el paquete que produce el flujo es un artefacto distinto del versionado que solo se publica como versión preliminar.

Qué falta

Alinear el resumen del guion de construcción que declara la evidencia con el archivo realmente versionado, y publicar como versión definitiva el mismo binario que se entrega.

5 Por dónde empezar

El orden que sugiero no es el de la tabla, sino el que evita rehacer trabajo.

1. **Primero lo que está en Modificar y afecta al documento.** Son correcciones cortas y son las que un revisor detecta enseguida: una cifra que no cuadra, una afirmación que el código no sostiene, una tabla con filas contradictorias. Cerrarlas primero sube mucho la calidad percibida por poco esfuerzo.
2. **Después lo que cambia números.** Toda corrección de medición obliga a reescribir tablas y a veces conclusiones. Hacerlo antes de redactar ahorra reescribir dos veces.
3. **Luego lo que está en Culminar y en Iniciar.** Es lo que más tiempo cuesta, así que conviene repartirlo por ámbito entre los cuatro y empezarlo en paralelo con lo anterior.
4. **Al final la redacción, las capturas y la publicación etiquetada.** Dependen de que los números ya sean los definitivos.
5. **Y desde el primer día, la integración.** Nada de acumular ramas. Cada cosa terminada se integra a la rama principal por solicitud de incorporación revisada por un compañero distinto del autor. Dejar la integración para el último día es la forma más frecuente de perder trabajo ya hecho.

6 Del repositorio al manuscrito

Lo que ustedes tienen en el repositorio no es el manuscrito: es el material del que sale el manuscrito. Esta sección explica cómo se traduce lo uno en lo otro, porque es donde los equipos suelen perder el trabajo que ya hicieron.

La regla que ordena todo

Cada afirmación no trivial del documento tiene que poder comprobarse abriendo el repositorio. Esa es la única regla, y de ella se derivan las demás. Si el documento dice que la cobertura es del ochenta por ciento, tiene que existir el comando que produce ese ochenta por ciento. Si dice que una campaña se ejecutó diez veces, tienen que estar los diez directorios de datos crudos. Si dice que un umbral se fijó antes de medir, la fecha del *commit* que lo fija tiene que ser anterior a la fecha de la ejecución.

Un evaluador externo no comprueba si ustedes trabajaron: comprueba si lo que afirman se sostiene. Y una afirmación que no se sostiene hace más daño que una ausencia reconocida, porque la ausencia es honesta y la afirmación falsa obliga al lector a desconfiar de todo lo demás. Los estándares de comunicación empírica en ingeniería de software insisten en este punto por encima de cualquier otro [1, 2].

De qué entregable sale cada parte del documento

Parte del manuscrito	De dónde sale
Contexto y problema	Los registros de decisión de arquitectura y el análisis del dominio. El problema tiene que estar cuantificado, no enunciado.
Arquitectura del sistema	Las vistas C4 en sus tres niveles, con la fuente versionada, y el contrato de interfaz. Cada tecnología que el diagrama nombra debe aparecer en el archivo de dependencias del componente al que se atribuye.
Datos y consistencia	El esquema, las migraciones y la evidencia de fragmentación, replicación y tolerancia a fallos. La posición en el teorema CAP se argumenta por agregado del dominio, no para todo el sistema.
Procesamiento paralelo	El trabajo distribuido con su curva de aceleramiento y el ajuste de la fracción no paralelizable. Si el resultado es una desaceleración, se reporta como tal y se explica.
Diseño experimental	El protocolo, los umbrales prerregistrados y el instrumental de medición. Los umbrales se declaran antes de medir y la fecha del <i>commit</i> lo demuestra [1].
Resultados y análisis	Los datos crudos, el análisis estadístico y las figuras. En comparaciones de rendimiento se usan pruebas no paramétricas y se reporta el tamaño del efecto junto al valor de significación, nunca el valor de significación solo [3, 4].
Evaluación de calidad	Las características medidas con su valor, su intervalo y su umbral, según el modelo de calidad de producto vigente [5]. Una característica que no cumple se consigna como no cumplida.
Amenazas a la validez	Todo lo que quedó en Modificar o Culminar y no se pudo cerrar, dicho con sus palabras y con su alcance real.
Ética	La reflexión referida al código de ética profesional [6], con el tratamiento de los datos personales que el sistema maneja.

Cinco errores que un revisor detecta en la primera lectura

1. **La medición que elige su propio alcance favorable.** Publicar una cobertura calculada sobre los archivos que las pruebas ya tocan, o una compuerta aplicada a las clases que ya pasan. La cifra sube y deja de describir el proyecto. Si hace falta excluir algo, se excluye y se justifica cada exclusión una por una.
2. **El número que no se reproduce.** Una cifra escrita a mano en una tabla se desactualiza en cuanto el código cambia. Toda tabla de resultados debería salir de un guion que lea el dato crudo y emita el bloque del documento.
3. **La prueba que no puede fallar.** Una prueba que pasa igual con el sistema apagado, con la aserción invertida o con la línea que debía verificar borrada, no prueba nada. Compruébenlo ustedes antes de publicarla: rompan algo y miren si se pone en rojo.
4. **El instrumento que altera lo que mide.** La instrumentación tiene coste, y ese coste entra en las cifras que se publican [7]. Conviene declarar en qué condiciones se midió y con qué instrumentación activa.
5. **El documento que va por detrás del repositorio.** El PDF versionado se desfasa del archivo fuente en cuanto alguien corrige el fuente y no recompila. Un trabajo del flujo automático que compile y compare cierra ese agujero para siempre.

Sobre la medición de un sistema distribuido

Medir un sistema de microservicios tiene dificultades propias que conviene nombrar en el documento en lugar de esquivarlas. La carga que se aplica debe ejercitar el camino crítico y no solo las lecturas públicas, porque son las operaciones que escriben las que entran en conflicto. La escalabilidad se mide con un método declarado y repetible, no con una corrida suelta [8]. Y las dificultades que los propios profesionales reportan al probar y monitorizar estos sistemas —la dependencia del entorno, el coste de levantar todo para una prueba, la fragilidad de las pruebas de extremo a extremo— son un marco legítimo para discutir las limitaciones del trabajo [9, 10].

7 Criterios de piso

Estos criterios no puntúan: se comprueban antes de calificar, y el incumplimiento de cualquiera de ellos deja la calificación en **CERO**.

Criterios de piso

1. **El PDF en el SGA con la dirección del repositorio.** Cada estudiante sube al SGA un documento PDF con carátula de identificación que muestre, en una sola línea y de forma claramente legible, la dirección del repositorio donde está el trabajo. Si esa dirección está mal escrita, el repositorio no es accesible, o no se cumple esta directriz, la calificación es CERO.
2. **El PDF se regenera desde el LaTeX.** Clono el repositorio, compilo el archivo principal y tiene que salir el mismo PDF. Las instrucciones de compilación —compilador, orden de comandos, archivo principal y paquetes necesarios— deben estar escritas en el README. Si el PDF no se genera de esa manera, la calificación es CERO.
3. **Todo integrado en la rama principal.** La entrega es un *commit* etiquetado de la rama principal. Si al clonar esa rama falta trabajo que ustedes consideran entregado porque vive en otra rama, la calificación es CERO.
4. **Cada uno responde por lo suyo.** En la defensa, cada integrante tiene que explicar el código y las mediciones que se le atribuyen: por qué están escritas así, qué pasa si se cambia tal cosa, de dónde salió tal número. El contenido que su autor declarado no sepa explicar se retira de la evaluación; si eso ocurre de forma sistemática en un integrante, su calificación es CERO. Esto aplica igual sea el código de un compañero, de un tutorial o de una herramienta automática: lo que se entrega, se entiende.

8 Lista de comprobación antes de entregar

1. Clonar el repositorio en un directorio vacío y compilar el documento siguiendo únicamente lo que dice el README. Si no sale el mismo PDF, todavía no está listo.
2. Ejecutar todas las suites de pruebas sobre ese clon limpio y comprobar que pasan.
3. Ejecutar todas las verificaciones de sumas y comprobar que pasan enteras.
4. Abrir el PDF y comprobar que no hay ninguna referencia ni cita sin resolver.
5. Recorrer las cifras del documento y comprobar, una por una, que cada una se reproduce con el comando que el propio documento indica.
6. Comprobar que ninguna rama va por delante de la principal.
7. Comprobar que el flujo automático está en verde en el *commit* que se entrega.

8. Crear la etiqueta de versión y la publicación con el documento y el instalable.
9. Subir al SGA el PDF con carátula y la dirección del repositorio en una sola línea.

Referencias

- [1] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.
- [2] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [3] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [4] A. Vargha and H. D. Delaney, “A critique and improvement of the CL common language effect size statistics of McGraw and Wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [5] International Organization for Standardization, “ISO/IEC 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model,” standard, ISO, Geneva, Switzerland, 2023.
- [6] Association for Computing Machinery, “ACM code of ethics and professional conduct.” <https://www.acm.org/code-of-ethics>, 2018.
- [7] Y. Hammad, A. Al-Said Ahmad, and P. Andras, “An empirical study on the performance overhead of code instrumentation in containerised microservices,” *Journal of Systems and Software*, vol. 230, p. 112573, 2025.
- [8] S. Henning and W. Hasselbring, “A configurable method for benchmarking scalability of cloud-native applications,” *Empirical Software Engineering*, vol. 27, no. 6, p. 143, 2022.
- [9] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, “Design, monitoring, and testing of microservices systems: The practitioners’ perspective,” *Journal of Systems and Software*, vol. 182, p. 111061, 2021.
- [10] M. Hui, L. Wang, H. Li, R. Yang, Y. Song, H. Zhuang, D. Cui, and Q. Li, “Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,” *Journal of Systems and Software*, vol. 220, p. 112232, 2025.

Anexo. Rúbrica de calificación

La tabla reproduce los entregables de la sección 3 con una columna adicional: el porcentaje de la calificación de la entrega que corresponde a cada uno. Los 30 entregables en **Hecho** siguen en Hecho y no se califican en esta entrega; los 17 restantes reparten el 100 % en proporción a la profundidad, la complejidad, la amplitud y el tiempo de trabajo que exige llevarlos a Hecho, con la calidad descrita en la sección 4, desde el estado en que se encontraban el domingo 6 de septiembre de 2026. Se califica el estado del repositorio en su último *commit* hasta el **viernes 18 de septiembre de 2026 a las 23:55**.

#	Entregable	Ámbito	Estado	% calificación
1	Refactorización en capas y principios SOLID	Arquitectura	Hecho	—
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho	—
3	Vistas de arquitectura C4 (tres niveles) versionadas como fuente	Arquitectura	Hecho	—
4	Contrato de interfaz OpenAPI	Arquitectura	Culminar	7,8 %
5	Esquema de base de datos y migraciones	Datos	Culminar	7,3 %
6	Persistencia distribuida operando con fragmentación y réplica	Datos	Hecho	—
7	Aplicación web funcional (construye y ejecuta)	Aplicaciones	Hecho	—
8	Aplicación móvil funcional	Aplicaciones	Hecho	—
9	Capacidades de dispositivo (cámara y notificaciones)	Aplicaciones	Hecho	—
10	Integración web + móvil + backend	Aplicaciones	Hecho	—
11	Pruebas de unidad e integración del backend	Pruebas	Hecho	—
12	Pruebas de contrato	Pruebas	Culminar	8,4 %
13	Pruebas de extremo a extremo	Pruebas	Modificar	7,8 %
14	Pruebas de carga	Pruebas	Culminar	8,4 %
15	Cobertura y complejidad reportadas con valores verificables	Pruebas	Modificar	7,3 %
16	Pipeline de integración con compuerta de calidad efectiva	Flujo	Hecho	—
17	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Modificar	5,6 %
18	Procesamiento paralelo	Medición	Hecho	—
19	Protocolo experimental e instrumental de medición	Medición	Hecho	—
20	Campaña ejecutada y totales reproducibles desde el dato crudo	Medición	Hecho	—
21	Análisis estadístico con tamaño de efecto e intervalos	Medición	Hecho	—
22	Datos crudos preservados	Medición	Modificar	6,1 %
23	Evaluación de calidad con las características medidas	Medición	Hecho	—
24	Manuscrito compilable, sin referencias indefinidas ni figuras huérfanas	Manuscrito	Hecho	—
25	Verificación del hash del artefacto documental	Manuscrito	Hecho	—
26	Bibliografía verificada con identificador persistente	Manuscrito	Modificar	2,2 %
27	Preguntas de investigación formuladas y respondidas	Manuscrito	Hecho	—
28	Amenazas a la validez tratadas por tipo y con mitigación	Manuscrito	Hecho	—
29	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho	—

#	Entregable	Ámbito	Estado	% calificación
30	Corrección explícita de afirmaciones previas no sostenidas	Manuscrito	Hecho	—
31	Integración de las ramas de trabajo a la principal	Flujo	Culminar	3,9 %
32	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho	—
33	Licencia y archivo de citación	Formalidades	Hecho	—
34	Actas de reunión	Formalidades	Culminar	2,8 %
35	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho	—
36	Capturas de la aplicación en release	Evidencia	Hecho	—
37	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Iniciar	5,0 %
38	Pruebas de unidad de la aplicación web	Pruebas	Modificar	7,3 %
39	Análisis estático de la aplicación web	Pruebas	Hecho	—
40	Publicación de imágenes en el registro prescrito	Flujo	Hecho	—
41	Provisión de tableros sin duplicación	Observabilidad	Hecho	—
42	Sumas de verificación de los datos crudos	Medición	Modificar	5,6 %
43	Todo archivo de evidencia citado existe en el árbol	Manuscrito	Hecho	—
44	Paquete móvil firmado y publicado en release	Aplicaciones	Modificar	5,6 %
45	Raíz del repositorio sin carpetas huérfanas	Formalidades	Hecho	—
46	Fijación de finales de línea para verificación reproducible	Medición	Culminar	5,0 %
47	Publicación etiquetada de la entrega	Flujo	Culminar	3,9 %
Total (17 entregables calificados)				100,0 %

Guía para culminar el Proyecto Fin de Curso

Equipo BCEL

Aplicaciones Distribuidas (ISR-701) · Séptimo semestre · Carrera de Ingeniería de Software
Facultad de Ciencias de la Computación y Diseño Digital · UTEQ · PPA 2026–2027

El proyecto

AcadTrace — Gestión académica con bitácora de auditoría encadenada

Repositorio: <https://github.com/LE023as/acadtrace>

Estado revisado sobre la rama main, *commit* 78b0d17 del 13 de septiembre de 2026, con 572 *commits* de historia.

Integrantes: Keyla Bedón, Juliana Emanuel, Ernesto Luna, Pedro Castro.

1 Para qué sirve esta guía

Esta guía responde a una sola pregunta: qué les falta para cerrar el Proyecto Fin de Curso con un manuscrito que se sostenga. Para responderla revisé el repositorio entero sobre el último *commit*, entregable por entregable, sin límite de fecha. No me quedé en comprobar que los archivos estuvieran: ejecuté las suites de pruebas, medí yo mismo la cobertura sin los filtros del proyecto, recalculé desde los datos crudos las cifras que el documento publica, ejecuté las verificaciones de sumas contando cuántas pasan y cuántas fallan, y comprobé en cada caso que las pruebas pudieran fallar rompiendo una aserción o el dato.

El resultado es la tabla de la sección 3: los **51 entregables** que el proyecto debía desarrollar, cada uno con su estado. De ellos, 29 están en Hecho y 22 necesitan trabajo. Ese es el alcance real de lo que queda.

Lectura del estado del proyecto

El trabajo correctivo de este equipo fue el más voluminoso del curso y buena parte de él es sustantivo: la bitácora de auditoría tiene hoy una sola forma canónica, que comprobé calculando el mismo vector patrón con las funciones de producción de los tres lenguajes; el bloqueo pesimista que el registro de decisión afirmaba existe de verdad; el certificado de reproducibilidad pasa doce de doce comprobaciones; los finales de línea están renormalizados con una compuerta que sabe fallar; y hay por fin una prueba que ejecuta el motor real de migraciones. Lo que impide cerrar el PFC es de una sola naturaleza, y es la que un revisor externo detecta primero: el documento afirma cosas que el repositorio no sostiene. La compuerta de cobertura mide seis clases escogidas a mano mientras el módulo está al 30 %; la matriz de calidad tiene filas duplicadas y contradictorias; las medianas publicadas son mil veces mayores que sus datos; el PDF compilado publica cifras que el propio equipo ya retractó en el archivo fuente; y el diagrama de arquitectura nombra una tecnología que el proyecto no usa.

2 Los cuatro estados

Estado	Qué significa exactamente
Hecho	El entregable está terminado, es correcto al cien por cien y está listo para usarse en la redacción del manuscrito. Se puede citar tal como está: la cifra que publica se reproduce, el archivo que nombra existe, la prueba que declara puede fallar.
Modificar	El entregable está terminado, pero no está correctamente desarrollado. No está listo para la redacción: usarlo tal como está introduciría en el manuscrito un error o una afirmación que el repositorio no sostiene.
Culminar	El desarrollo está iniciado y hay que continuarlo. La pieza existe y funciona en parte, pero le falta lo que la haría utilizable como evidencia.
Iniciar	El entregable no está presente en el proyecto. Hay que construirlo desde cero.

La diferencia entre Modificar y Culminar, que es la que más se confunde

Un entregable en **Modificar** ya está construido y el trabajo que queda es de corrección: cambiar un número que no cuadra, quitar una afirmación que el código no sostiene, retirar un filtro que infla una medición. Suele costar poco tiempo y suele ser lo que más daño hace si se deja, porque un revisor externo lo detecta abriendo el repositorio y pierde la confianza en todo lo demás.

Un entregable en **Culminar** tiene el trabajo a medias y lo que queda es construcción: ejecutar una campaña que no se ejecutó, escribir el verificador que falta, capturar la evidencia que nadie capturó. Cuesta más tiempo, pero mientras está a medias no engaña a nadie.

Por eso el orden de ataque que propongo más adelante empieza por lo que está en Modificar y no por lo que parece más grande.

3 Todos los entregables, con su estado

La tabla recoge todo lo que el proyecto debía desarrollar, incluido lo que ya está resuelto. La columna de estado es la que manda: lo que aparece en Hecho no hay que volver a tocarlo, y puede citarse en el manuscrito tal como está.

#	Entregable	Ámbito	Estado
1	Refactorización en capas y principios SOLID	Arquitectura	Hecho
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho
3	Vistas de arquitectura C4 (tres niveles)	Arquitectura	Modificar
4	Contrato de interfaz OpenAPI	Arquitectura	Modificar
5	Esquema de base de datos y migraciones	Datos	Culminar
6	Persistencia distribuida operando	Datos	Modificar
7	Aplicaciones web funcionales (cuatro interfaces)	Aplicaciones	Hecho
8	Aplicación móvil funcional	Aplicaciones	Hecho
9	Capacidades de dispositivo (biometría y notificaciones)	Aplicaciones	Hecho
10	Integración web + móvil + backend	Aplicaciones	Hecho
11	Pruebas de unidad e integración del backend Java	Pruebas	Hecho
12	Pruebas del microservicio en Python con cobertura medida	Pruebas	Modificar
13	Pruebas de contrato verificadas contra servidor real	Pruebas	Hecho
14	Pruebas de carga	Pruebas	Hecho

#	Entregable	Ámbito	Estado
15	Pipeline de integración con dependencias declaradas	Flujo	Hecho
16	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Hecho
17	Procesamiento paralelo	Medición	Hecho
18	Bitácora de auditoría encadenada (implementación en Python)	Arquitectura	Hecho
19	Protocolo experimental e instrumental de medición	Medición	Hecho
20	Datos crudos preservados	Medición	Hecho
21	Análisis estadístico con tamaño de efecto e intervalos	Medición	Modificar
22	Evaluación de calidad con las características medidas	Medición	Modificar
23	Manuscrito consolidado, compilable y sin referencias indefinidas	Manuscrito	Hecho
24	Bibliografía verificada con identificador persistente	Manuscrito	Hecho
25	Preguntas de investigación formuladas y respondidas	Manuscrito	Hecho
26	Amenazas a la validez con mitigación declarada	Manuscrito	Hecho
27	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho
28	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho
29	Licencia y archivo de citación	Formalidades	Hecho
30	Actas de reunión	Formalidades	Hecho
31	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho
32	Capturas de la aplicación en release	Evidencia	Modificar
33	Paquete móvil publicado en release	Aplicaciones	Culminar
34	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Hecho
35	Instalación reproducible de todas las interfaces desde un clon limpio	Flujo	Modificar
36	Pruebas de extremo a extremo sobre navegador	Pruebas	Modificar
37	Pruebas de integración sobre contenedores efímeros	Pruebas	Modificar
38	Cobertura reportada con una sola cifra coherente	Pruebas	Culminar
39	Dependencias con versiones existentes e instalables	Flujo	Hecho
40	Publicación de todas las imágenes del sistema	Flujo	Modificar
41	Despliegue que falla cuando debe fallar	Flujo	Hecho
42	Mecanismo de auditoría único y coherente en todo el sistema	Arquitectura	Modificar
43	Verificador experimental que comprueba la cadena de producción	Medición	Modificar
44	Disparador de inmutabilidad efectivamente aplicado a la base	Datos	Modificar
45	Restricción de privilegios de modificación sobre la bitácora	Datos	Culminar
46	Gestión de secretos fuera del árbol versionado	Formalidades	Culminar
47	Certificado de reproducibilidad consistente con los datos	Medición	Hecho
48	Conjunto de resultados de carga único y declarado	Medición	Modificar
49	Fijación de finales de línea para verificación reproducible	Medición	Hecho
50	Mapa en el README entre las carpetas del proyecto y la estructura prescrita	Formalidades	Culminar
51	Integración de todas las ramas de trabajo a la principal	Flujo	Culminar

Resumen

Estado	Entregables	Proporción
Culminar	7	14 %
Modificar	15	29 %
Hecho	29	57 %
Total	51	100 %

El trabajo pendiente se reparte así por ámbito: Datos (4); Pruebas (4); Medición (4); Arquitectura (3); Flujo (3); Formalidades (2); Evidencia (1); Aplicaciones (1). Esa distribución sirve para repartirse el trabajo sin que dos personas se pisen.

4 El trabajo que queda, entregable por entregable

De cada entregable pendiente doy tres cosas. **La situación** es lo que encontré al revisar, con el archivo y el dato concreto, para que no tengan que adivinar de qué hablo. **Qué falta** es la acción que cierra el punto. Y el **estado** dice de dónde parte cada uno.

Entregables en Culminar — el desarrollo está a medias

5. Esquema de base de datos y migraciones

Datos · **Culminar**

La situación. La causa se arregló pero el aviso sigue silenciado. Listé el directorio de migraciones y hoy hay nueve, sin ningún número repetido; rastreando el historial confirmé que la migración que colisionaba se reenumeró. También comprobé que el archivo de configuración no desactiva nada: declara el motor activo, su línea base y su esquema, sin bandera de validación. Sin embargo la variable que se introdujo para que el arranque no se detuviera sigue viva en la línea 27 del archivo de composición, que es el archivo con el que se levanta el sistema y con el que despliega el flujo. Mientras esa línea siga ahí, el motor no avisará de ninguna inconsistencia futura y el criterio de que el arranque pase con la validación activada no queda demostrado en ningún sitio.

Qué falta

Borrar esa línea del archivo de composición y dejar constancia de un arranque correcto con la validación activada, tanto sobre base nueva como sobre base ya migrada.

33. Paquete móvil publicado en release

Aplicaciones · **Culminar**

La situación. Se preparó el andamiaje de firma pero el instalable publicado sigue siendo el de depuración. El archivo de construcción tiene ahora un bloque que crea la configuración de entrega leyendo cuatro variables del entorno o de un archivo de propiedades, y el tipo de entrega la asigna. Pero esa configuración es condicional: si el almacén no está presente, la búsqueda devuelve nulo y la construcción no falla, solo produce un paquete sin firmar. Y el artefacto que se publica no pasa por ahí: el paquete de 19.992.081 bytes está construido con la variante de depuración según su propio índice, y analicé su bloque de firma y el titular del certificado sigue siendo el genérico de depuración. El flujo tampoco cierra el punto: los dos trabajos que tocan el móvil compilan y suben el paquete de depuración, no hay ningún paso de firma ni ninguna condición que falle si no se puede firmar.

Qué falta

Generar el instalable con la variante de entrega firmada con un certificado propio suministrado por secretos del flujo, hacer que el trabajo falle si el almacén no está disponible, y documentar en el índice la verificación de la firma.

38. Cobertura reportada con una sola cifra coherente

Pruebas · **Culminar**

La situación. La compuerta sigue midiendo clases escogidas a mano y las cifras siguen sin cuadrar. Abrí los tres archivos de proyecto: en dos de ellos la regla es de elemento clase con una lista de seis y siete clases respectivamente al 70 % de instrucciones, y solo el tercero mide el módulo entero. Recalculé la cobertura real sumando las columnas de los informes versionados y obtuve resultados que se contradicen dentro del propio repositorio: para el módulo principal un informe da 29,87 % y otro 26,24 %; para el de secretaría un informe da 35,83 % y otro 71,25 %; el tercero da 73,05 % y del módulo móvil no hay ningún informe. El documento sí retiró las cifras falsas anteriores y ahora escribe «cobertura parcial no consolidada», pero eso significa que ningún módulo publica su cifra real, y la afirmación sin respaldo del 90,7 % sigue en el texto junto con la descripción de la compuerta como estricta al 70 %.

Qué falta

Sustituir en los dos archivos de proyecto la regla de lista de clases por una de módulo entero con exclusiones justificadas, publicar una sola cifra medida por módulo eliminando los informes duplicados, y borrar del texto la cifra sin respaldo.

45. Restricción de privilegios de modificación sobre la bitácora

Datos · **Culminar**

La situación. La mitad que faltaba sí se hizo: la migración crea el rol de aplicación de forma idempotente sin privilegios de superusuario, le concede solo lectura e inserción sobre la bitácora y revoca modificación, borrado y truncado tanto al rol como al público, repitiendo la revocación después de la concesión general para que no la anule; y la prueba comprueba en el motor que el rol existe tras migrar, desactiva el disparador y verifica que las dos operaciones fallan por permiso denegado. Lo que no se hizo es que la aplicación se conecte con ese rol: el archivo de configuración toma el usuario de una variable cuyo valor por omisión es el propietario del esquema, y tanto el archivo de composición como el de ejemplo lo fijan así; buscando en el repositorio, el rol restringido solo aparece en la migración y en esa prueba. El experimento es correcto pero el sistema que se levanta sigue escribiendo la bitácora como superusuario.

Qué falta

Cambiar el usuario de conexión al rol restringido en el archivo de composición y en el de ejemplo, suministrar su contraseña por variable en lugar de dejarla escrita en la migración, y comprobar al arrancar que el usuario conectado no puede modificar la bitácora.

46. Gestión de secretos fuera del árbol versionado

Formalidades · **Culminar**

La situación. Verifiqué que la lista blanca desapareció: el archivo de configuración del detector contiene hoy solo la extensión del conjunto de reglas por omisión, sin exclusiones ni rutas permitidas, y el flujo lo ejecuta de verdad con historial completo; el documento de rotación declara la fecha e inventario por secreto. Eso resolvió la compuerta falseada. Lo que no resolvieron es el enunciado del punto: ejecuté el detector sobre 856 commits, primero con la configuración del repositorio y después con la de por omisión, y ambas devuelven exactamente 27 fugas, es decir el historial nunca se reescribió; agrupadas por valor son once apariciones de una contraseña, diez de una clave simétrica, cuatro de una clave de cifrado y dos casos aislados, con fechas del 25 de mayo al 12 de septiembre. Además el barrido del árbol de trabajo todavía arroja tres hallazgos: dos en un paquete minificado versionado y uno en una clase de prueba.

Qué falta

Reescribir el historial para eliminar los 27 hallazgos, retirar del control de versiones el directorio compilado que contiene el paquete minificado, anotar la constante de prueba, y dejar que

el detector con configuración por omisión salga en cero.

50. Mapa en el README entre las carpetas del proyecto y la estructura prescrita Formalidades · **Culminar**

La situación. Probé los nueve comandos de la tabla sobre un clon limpio. Las nueve carpetas existen y dos de los cuatro comandos rotos se arreglaron: el guion que no tenía punto de entrada ahora sí lo tiene y termina con código cero imprimiendo el bloque inicial y el resultado de la verificación, y dos de los arrancadores están en la ruta correcta y son ejecutables. Pero quedan tres filas que no producen lo que anuncian: dos fallan con permiso denegado porque sus arrancadores están versionados sin el bit de ejecución, a diferencia de sus gemelos, y de hecho el propio flujo les da permisos antes de usarlos; y la tercera aborta con tres errores de recolección por una variable de entorno que el índice no menciona y que el flujo sí define.

Qué falta

Marcar como ejecutables los dos arrancadores en el propio control de versiones, y añadir a la fila que falla la variable de entorno que necesita o darle un valor por omisión en el código.

51. Integración de todas las ramas de trabajo a la principal

Flujo · **Culminar**

La situación. Contrasté cada rama remota con la principal y el trabajo sin fusionar bajó mucho: de veintiocho commits en cinco ramas quedan seis en tres, y las cinco ramas personales ya no van por delante. Lo que queda fuera no es accesorio: incluye la exclusión de plantillas y objetos de transferencia de la medición de cobertura y una limpieza final que afecta precisamente a las métricas oficiales. Sobre las aprobaciones, comparé el autor de cada commit de fusión con los autores de los commits de la rama fusionada y las solicitudes las sigue confirmando el propio autor: de las seis que el documento tabula como validadas por un integrante distinto, cinco fueron fusionadas por el mismo autor de la rama y solo una por otra persona.

Qué falta

Fusionar o cerrar explícitamente los seis commits de las tres ramas pendientes, y sustituir la columna de validación de la tabla por el revisor que el historial acredite, o retirar la afirmación.

Entregables en Modificar — están hechos pero no son utilizables como evidencia

3. Vistas de arquitectura C4 (tres niveles)

Arquitectura · **Modificar**

La situación. Abrí la imagen del nivel de contenedores que el documento inserta y el contenedor de portales web aparece rotulado con una biblioteca de interfaz que el proyecto no usa; la misma cadena está en el fuente vectorial, en su copia, en el fuente del diagrama y en el índice de diagramas. Contrasté contra los archivos de dependencias de los cuatro componentes a los que se atribuye y ninguno la declara: los cuatro usan React 19 con Vite. El diagrama describe un sistema distinto del que se entrega, y es la figura que el manuscrito usa para presentar la arquitectura.

Qué falta

Corregir la tecnología en el fuente del diagrama, regenerar el vectorial y la imagen, y actualizar las dos copias versionadas.

4. Contrato de interfaz OpenAPI

Arquitectura · **Modificar**

La situación. El contrato declara seis rutas y cuatro de ellas usan una palabra que el código no usa. Busqué esa cadena en todo el código y en los cuatro frontales, excluyendo dependencias y

compilados, y devuelve cero coincidencias; lo que existe son nueve controladores anotados con la misma palabra en singular masculino. Solo dos rutas del contrato responden a algo real. Un cliente generado desde este contrato llamaría a cuatro direcciones inexistentes. Comprobé además que en el flujo no hay ningún paso que mencione el contrato, de modo que nada compara el documento versionado con el que sirve el servicio.

Qué falta

Renombrar las cuatro rutas del contrato a la forma que el código usa, y añadir al flujo un paso que levante el servicio, descargue el contrato que sirve y lo compare con el versionado.

6. Persistencia distribuida operando

Datos · [Modificar](#)

La situación. Enumeré los dieciséis servicios del archivo de composición y no hay ninguno de base de datos: todos apuntan a una instancia externa en la nube. El único motor de base de datos declarado en el repositorio está en un archivo de composición secundario y es una instancia suelta. Frente a eso, el documento afirma que «el clúster de persistencia distribuida opera bajo una topología de tres instancias redundantes sincronizadas mediante el algoritmo de consenso Raft», con particionamiento vertical y horizontal, y el registro de decisión correspondiente decide un motor distribuido con ese algoritmo; no hay una sola aparición de ese motor ni de sus alternativas en el repositorio. Lo único con ese algoritmo es el coordinador que sirve para elección de líder en otro servicio, no para la persistencia. El propio diagrama de contenedores que el documento inserta dibuja una instancia única, contradiciendo el texto.

Qué falta

Reescribir la sección de persistencia distribuida y su registro de decisión para describir la instancia multiesquema que realmente se levanta, o incorporar al archivo de composición el clúster de tres nodos que se afirma y medir algo de su consenso.

12. Pruebas del microservicio en Python con cobertura medida

Pruebas · [Modificar](#)

La situación. Medí la cobertura exactamente con el comando que declara el flujo: con las 102 pruebas en verde el total es 78,47 % sobre 2.378 sentencias; volviendo a generar el informe sobre la misma medición pero excluyendo los archivos de prueba, el denominador baja a 1.536 sentencias y la cifra honesta queda en 67 %. Es decir, 842 sentencias del denominador son código de prueba, cubierto al cien por cien por definición. Revisé el archivo de configuración de la medición y sus exclusiones cubren migraciones, código generado y comandos de gestión, pero no los archivos de prueba. La consecuencia es concreta: la compuerta del flujo exige 70 % y pasa con 78,47 %, mientras que con la medición honesta al 67 % la construcción rompería.

Qué falta

Añadir los patrones de archivos de prueba a las exclusiones de la medición y ajustar la suite hasta superar el umbral con esa medición honesta.

21. Análisis estadístico con tamaño de efecto e intervalos

Medición · [Modificar](#)

La situación. Recaliculé las medianas directamente desde el archivo de concurrencia, que tiene 160 filas, cuarenta por mecanismo, y la columna de latencia mediana da 0,001, 0,002, 0,007 y 0,010 ms, mientras la tabla de estadística inferencial del documento publica 1,49, 4,31, 7,10 y 9,62 ms: entre 962 y 2.155 veces mayores, de modo que el cambio de unidad sigue sin aplicarse. El documento se contradice consigo mismo en dos sitios: la conclusión de la sección de cierre afirma una mediana de 0,007 ms frente a un escenario base de 0,003 ms, que sí está en el orden del dato crudo, y la figura de cajas que el mismo documento inserta tiene el eje rotulado en milisegundos con rango de 0,0025 a 0,0225, mil veces por debajo de la tabla que la acompaña. Ejecuté además

el guion de experimentos de principio a fin sobre un clon limpio y su propio resumen imprimió medianas de 0,023 a 0,025 ms con un contraste de p igual a 0,52 y tamaño de efecto 0,479, cuando la tabla publica p menor que diez elevado a menos quince y tamaños entre 0,959 y 1,000.

Qué falta

Regenerar la tabla de latencias con un guion que lea el archivo crudo y emita el bloque del documento, declarando la unidad real, y descartar los contrastes y tamaños de efecto que la ejecución actual no sostiene.

22. Evaluación de calidad con las características medidas

Medición · **Modificar**

La situación. El archivo de mediciones sí se corrigió: recalculé sus veinte ventanas y la columna de disponibilidad de producción vale ahora «No disponible» en las veinte filas, y la de éxito da 100 % en las diez ventanas nominales y entre 12,78 % y 34,58 % en las diez de estrés, etiquetado además como perfil no validado; desaparecieron las disponibilidades presentadas como si fueran la medida buena. El problema es que en la matriz de calidad del documento quedaron las filas antiguas junto a las nuevas: la característica de disponibilidad aparece dos veces, la segunda afirmando cien por cien nominal y cero por ciento en estrés, y la de fiabilidad aparece también dos veces con estados distintos; el título de la tabla sigue prometiendo datos cien por cien medidos y el párrafo que la introduce repite esa promesa, cuando la propia fila nueva declara que no acredita disponibilidad de producción. Tampoco existe ningún comando que derive la tabla desde el archivo de mediciones.

Qué falta

Eliminar las filas duplicadas heredadas, corregir el título y el párrafo que prometen datos cien por cien medidos, y añadir un guion que emita la tabla directamente desde el archivo de mediciones.

32. Capturas de la aplicación en release

Evidencia · **Modificar**

La situación. Abrí las imágenes publicadas y el material ajeno sigue ahí, sin un solo cambio desde el 10 de septiembre. Una de ellas continúa siendo un logotipo genérico de caja tridimensional sobre fondo blanco, sin relación alguna con el proyecto. Otra sigue siendo un boceto de arquitectura fabricado, y al abrirlo confirmé que su tabla de topología atribuye un marco de Node a un servicio y un marco de Python a otro, cuando ambos están escritos en Java con Spring Boot según sus propios archivos de proyecto; la imagen muestra además cuadros de glifo ausente donde deberían ir iconos, señal de que es una página renderizada y no una captura. Comprobé también la segunda mitad del criterio: busqué las diez imágenes de esa carpeta en las inclusiones del documento y en todos los archivos de texto, y ninguna está referenciada desde ningún sitio.

Qué falta

Borrar las dos imágenes ajenas, sustituirlas por capturas reales de los portales, y referenciar desde el texto cada imagen que se conserve.

35. Instalación reproducible de todas las interfaces desde un clon limpio

Flujo · **Modificar**

La situación. Verifiqué que los cuatro pasos de compilación de interfaz usan ya la instalación reproducible que exige el archivo de bloqueo, que era el límite anterior. Comprobé que funciona partiendo de copias limpias sin carpetas de dependencias: las cuatro interfaces instalan con código cero y las cuatro completan su compilación de producción. Lo que impide el cierre es que tres de las cuatro instalaciones siguen pasando la bandera que relaja los conflictos entre dependencias declaradas, lo que enmascara incompatibilidades reales en lugar de resolverlas, que el módulo móvil obtiene su arrancador por red en cada ejecución en vez de usar el archivo versionado, y que en la raíz hay un archivo de bloqueo huérfano sin su archivo de proyecto.

Qué falta

Quitar la bandera que relaja los conflictos y resolver las incompatibilidades que aparezcan, usar el arrancador versionado en el módulo móvil, y borrar el archivo de bloqueo huérfano de la raíz.

36. Pruebas de extremo a extremo sobre navegadorPruebas · **Modificar**

La situación. Las pruebas mejoraron mucho y la evidencia no. Verifiqué que la suite antigua desapareció y en su lugar hay una real: autentica con credenciales del entorno, navega los módulos de calificaciones y asistencia por rol y texto reales, recorre cursos, trimestres y semanas hasta hallar una calificación modificable, la altera y restaura el valor original; el flujo la ejercita en un trabajo que valida antes que las cinco variables estén definidas. Las ejecuté contra una dirección sin aplicación y las seis fallan con conexión rechazada, que es el comportamiento correcto. Lo que no se tocó es la evidencia publicada: abrí las diez capturas del directorio correspondiente y son páginas escritas a mano, texto negro sobre fondo blanco, sin ningún elemento de la interfaz real del proyecto.

Qué falta

Sustituir las diez capturas por artefactos producidos por la propia suite contra el sistema levantado, y conservar en el árbol el informe de una ejecución en verde.

37. Pruebas de integración sobre contenedores efímerosPruebas · **Modificar**

La situación. Conté cuatro suites que levantan una base de datos efímera y comprobé que la nueva cierra el hueco que había: configura el motor de migraciones apuntando a la ubicación real de producción, parte de la línea base y exige que consten aplicadas con éxito las nueve versiones siguientes en el historial oficial. Verifiqué que dos de las cuatro no están condicionadas y fallan de forma dura si no hay contenedores. Como la política de red bloquea aquí la descarga de imágenes, reproduce el contenido de la suite nueva sobre un motor local y confirmé que las migraciones se aplican, que el disparador queda creado y que rechaza modificación y borrado. El límite es que la suite nueva sí se desactiva en silencio cuando no hay contenedores y aquí se reportó omitida dos de dos, y que la suite de concurrencia sigue construyendo sus tablas a mano en lugar de migrarlas.

Qué falta

Hacer que la suite de concurrencia migre su esquema en vez de escribirlo a mano, y dejar constancia de una ejecución en verde de las cuatro suites en el flujo.

40. Publicación de todas las imágenes del sistemaFlujo · **Modificar**

La situación. Comprobé que la matriz cubre ahora los cinco servicios con archivo de construcción propio y que los cinco archivos existen en las rutas declaradas, de modo que los dos que quedaban sin publicar ya están incluidos; y que el trabajo se dispara también en ramas y solicitudes de incorporación construyendo sin publicar, con lo que toda solicitud ejercita la construcción. Lo que impide el cierre son dos cosas que verifiqué abriendo el archivo tal como está hoy: la publicación duplicada que un commit había eliminado fue reintroducida por una fusión posterior y sigue presente, de modo que el paso construye y publica por segunda vez la imagen de un servicio bajo un nombre distinto al de la matriz, y además está dentro de la matriz, con lo que se ejecuta una vez por cada uno de los cinco servicios; y la condición del trabajo permite que se ejecute y termine en verde en ramas y solicitudes aunque los trabajos de prueba hayan fallado.

Qué falta

Eliminar el paso duplicado que la fusión reintrodujo y exigir en todas las ramas, no solo en la principal, que ningún trabajo de prueba haya fallado.

42. Mecanismo de auditoría único y coherente en todo el sistemaArquitectura · **Modificar**

La situación. Comprobé la unificación con un vector patrón y no con la palabra del documento: tomé los once campos que fija la prueba canónica de Java y los calculé con las funciones de producción de Python, obteniendo carácter por carácter el mismo objeto canónico y el mismo hash que afirman las pruebas de los dos servicios en Java, es decir una sola forma canónica realmente compartida. Verifiqué que esa prueba puede fallar insertando un campo adicional, tras lo cual se puso en rojo. Confirmé también que el bloqueo pesimista existe ahora en los tres servicios y que el reloj lógico se siembra desde la fila persistida, de modo que ya no vuelve a uno tras un reinicio; el reloj vectorial dejó de ser inerte porque cada servicio funde el vector persistido antes de incrementar su componente. Lo que impide el cierre es que siguen existiendo tres implementaciones separadas del mismo contrato, sincronizadas únicamente por ese vector patrón y no por una biblioteca común, y que la prueba de concurrencia construye su esquema a mano en lugar de migrarlo.

Qué falta

Extraer la forma canónica a una biblioteca común importada por los tres servicios, o dejar constancia en el documento de que la sincronización se sostiene solo sobre el vector patrón y de qué la protege.

43. Verificador experimental que comprueba la cadena de producciónMedición · **Modificar**

La situación. Ejecuté la suite en un entorno limpio y sus seis casos pasan: acepta una cadena global válida de dos eslabones, rechaza un contenido canónico manipulado, un hash anterior roto, un reloj lógico no monotónico y una edición directa del estado académico, y comprueba por identidad de objeto que el adaptador es exactamente el verificador de producción. Comprobé que puede fallar sustituyendo por una condición siempre falsa la comparación entre el hash almacenado y el recalculado, tras lo cual la suite pasó a uno fallido y cinco correctos. Verifiqué también que el guion de experimentos importa el cálculo de hash, la forma canónica y el verificador desde el paquete de producción, sin reimplementación paralela. El límite es que la cadena que se verifica se construye sintéticamente en memoria y no se lee de una base real, y que la salida del adaptador afirma integridad del cien por cien tras validar un único eslabón testigo que él mismo acaba de fabricar.

Qué falta

Hacer que el verificador lea una cadena real de la base en lugar de fabricarla, y sustituir el mensaje de integridad total por el resultado real de lo que se verificó.

44. Disparador de inmutabilidad efectivamente aplicado a la baseDatos · **Modificar**

La situación. Leí la prueba y comprobé que ya no depende de ninguna variable inexistente ni envuelve la llamada a las migraciones en un bloque que se trague errores: afirma primero que no existen ni el historial ni el disparador, después ejecuta el motor real apuntando a la ubicación de producción desde la línea base sin capturar excepciones, y exige que las nueve versiones siguientes consten aplicadas y que el disparador exista con su definición completa; luego inserta un registro testigo y comprueba que modificación y borrado se rechazan y que el texto original sobrevive. Como aquí no hay entorno de contenedores, reproduje su contenido sobre un motor local y las nueve migraciones se aplicaron sin error, el disparador quedó definido y rechazó las dos operaciones. El límite es que la prueba se desactiva en silencio cuando no hay contenedores, y que su archivo de

partida es un esquema reducido escrito a mano y no la línea base de producción.

Qué falta

Hacer que la prueba falle en lugar de omitirse cuando no hay contenedores disponibles, y partir de la línea base real de producción en vez de un esquema reducido.

48. Conjunto de resultados de carga único y declarado

Medición · **Modificar**

La situación. Recalculé desde los archivos crudos y siguen conviviendo cinco juegos distintos del mismo tipo de escenario, con 12.994, 12.236, 13.606, 2.419 y 106.735 peticiones. El equipo sí corrigió la cifra inventada: la anterior desapareció del archivo fuente y ahora se declara oficial la de 12.994, y comprobé que esa fila y sus percentiles se reproducen exactamente desde su archivo, con 0 fallos, 43,537580 peticiones por segundo y percentiles de 6, 440 y 850 ms. Pero la contradicción sigue dentro del documento y hoy es peor: en la matriz de calidad la fila de disponibilidad aparece dos veces seguidas, la nueva con la cifra oficial y justo debajo la vieja, no borrada, con la cifra retractada; lo mismo con la fila de fiabilidad. Y el PDF compilado, con fecha anterior al archivo fuente, sigue publicando las cifras retractadas en siete lugares, incluidos el pie de una figura y la tabla comparativa. Tampoco existe ningún guion que regenere las cifras del documento desde el juego elegido.

Qué falta

Borrar las filas duplicadas de la matriz, retirar o marcar como no oficiales los cuatro juegos que no son el declarado, añadir un guion que derive cada cifra publicada desde ese único archivo, y recompilar el PDF.

5 Por dónde empezar

El orden que sugiero no es el de la tabla, sino el que evita rehacer trabajo.

1. **Primero lo que está en Modificar y afecta al documento.** Son correcciones cortas y son las que un revisor detecta enseguida: una cifra que no cuadra, una afirmación que el código no sostiene, una tabla con filas contradictorias. Cerrarlas primero sube mucho la calidad percibida por poco esfuerzo.
2. **Después lo que cambia números.** Toda corrección de medición obliga a reescribir tablas y a veces conclusiones. Hacerlo antes de redactar ahorra reescribir dos veces.
3. **Luego lo que está en Culminar y en Iniciar.** Es lo que más tiempo cuesta, así que conviene repartirlo por ámbito entre los cuatro y empezarlo en paralelo con lo anterior.
4. **Al final la redacción, las capturas y la publicación etiquetada.** Dependen de que los números ya sean los definitivos.
5. **Y desde el primer día, la integración.** Nada de acumular ramas. Cada cosa terminada se integra a la rama principal por solicitud de incorporación revisada por un compañero distinto del autor. Dejar la integración para el último día es la forma más frecuente de perder trabajo ya hecho.

6 Del repositorio al manuscrito

Lo que ustedes tienen en el repositorio no es el manuscrito: es el material del que sale el manuscrito. Esta sección explica cómo se traduce lo uno en lo otro, porque es donde los equipos suelen perder el trabajo que ya hicieron.

La regla que ordena todo

Cada afirmación no trivial del documento tiene que poder comprobarse abriendo el repositorio. Esa es la única regla, y de ella se derivan las demás. Si el documento dice que la cobertura es del ochenta por ciento, tiene que existir el comando que produce ese ochenta por ciento. Si dice que una campaña se ejecutó diez veces, tienen que estar los diez directorios de datos crudos. Si dice que un umbral se fijó antes de medir, la fecha del *commit* que lo fija tiene que ser anterior a la fecha de la ejecución.

Un evaluador externo no comprueba si ustedes trabajaron: comprueba si lo que afirman se sostiene. Y una afirmación que no se sostiene hace más daño que una ausencia reconocida, porque la ausencia es honesta y la afirmación falsa obliga al lector a desconfiar de todo lo demás. Los estándares de comunicación empírica en ingeniería de software insisten en este punto por encima de cualquier otro [1, 2].

De qué entregable sale cada parte del documento

Parte del manuscrito	De dónde sale
Contexto y problema	Los registros de decisión de arquitectura y el análisis del dominio. El problema tiene que estar cuantificado, no enunciado.
Arquitectura del sistema	Las vistas C4 en sus tres niveles, con la fuente versionada, y el contrato de interfaz. Cada tecnología que el diagrama nombra debe aparecer en el archivo de dependencias del componente al que se atribuye.
Datos y consistencia	El esquema, las migraciones y la evidencia de fragmentación, replicación y tolerancia a fallos. La posición en el teorema CAP se argumenta por agregado del dominio, no para todo el sistema.
Procesamiento paralelo	El trabajo distribuido con su curva de aceleramiento y el ajuste de la fracción no paralelizable. Si el resultado es una desaceleración, se reporta como tal y se explica.
Diseño experimental	El protocolo, los umbrales prerregistrados y el instrumental de medición. Los umbrales se declaran antes de medir y la fecha del <i>commit</i> lo demuestra [1].
Resultados y análisis	Los datos crudos, el análisis estadístico y las figuras. En comparaciones de rendimiento se usan pruebas no paramétricas y se reporta el tamaño del efecto junto al valor de significación, nunca el valor de significación solo [3, 4].
Evaluación de calidad	Las características medidas con su valor, su intervalo y su umbral, según el modelo de calidad de producto vigente [5]. Una característica que no cumple se consigna como no cumplida.
Amenazas a la validez	Todo lo que quedó en Modificar o Culminar y no se pudo cerrar, dicho con sus palabras y con su alcance real.
Ética	La reflexión referida al código de ética profesional [6], con el tratamiento de los datos personales que el sistema maneja.

Cinco errores que un revisor detecta en la primera lectura

1. **La medición que elige su propio alcance favorable.** Publicar una cobertura calculada sobre los archivos que las pruebas ya tocan, o una compuerta aplicada a las clases que ya pasan. La cifra sube y deja de describir el proyecto. Si hace falta excluir algo, se excluye y se justifica cada exclusión una por una.
2. **El número que no se reproduce.** Una cifra escrita a mano en una tabla se desactualiza en cuanto el código cambia. Toda tabla de resultados debería salir de un guion que lea el dato crudo y emita el bloque del documento.

3. **La prueba que no puede fallar.** Una prueba que pasa igual con el sistema apagado, con la aserción invertida o con la línea que debía verificar borrada, no prueba nada. Compruébenlo ustedes antes de publicarla: rompan algo y miren si se pone en rojo.
4. **El instrumento que altera lo que mide.** La instrumentación tiene coste, y ese coste entra en las cifras que se publican [7]. Conviene declarar en qué condiciones se midió y con qué instrumentación activa.
5. **El documento que va por detrás del repositorio.** El PDF versionado se desfasa del archivo fuente en cuanto alguien corrige el fuente y no recompila. Un trabajo del flujo automático que compile y compare cierra ese agujero para siempre.

Sobre la medición de un sistema distribuido

Medir un sistema de microservicios tiene dificultades propias que conviene nombrar en el documento en lugar de esquivarlas. La carga que se aplica debe ejercitar el camino crítico y no solo las lecturas públicas, porque son las operaciones que escriben las que entran en conflicto. La escalabilidad se mide con un método declarado y repetible, no con una corrida suelta [8]. Y las dificultades que los propios profesionales reportan al probar y monitorizar estos sistemas —la dependencia del entorno, el coste de levantar todo para una prueba, la fragilidad de las pruebas de extremo a extremo— son un marco legítimo para discutir las limitaciones del trabajo [9, 10].

7 Criterios de piso

Estos criterios no puntúan: se comprueban antes de calificar, y el incumplimiento de cualquiera de ellos deja la calificación en **CERO**.

Criterios de piso

1. **El PDF en el SGA con la dirección del repositorio.** Cada estudiante sube al SGA un documento PDF con carátula de identificación que muestre, en una sola línea y de forma claramente legible, la dirección del repositorio donde está el trabajo. Si esa dirección está mal escrita, el repositorio no es accesible, o no se cumple esta directriz, la calificación es CERO.
2. **El PDF se regenera desde el LaTeX.** Clono el repositorio, compilo el archivo principal y tiene que salir el mismo PDF. Las instrucciones de compilación —compilador, orden de comandos, archivo principal y paquetes necesarios— deben estar escritas en el README. Si el PDF no se genera de esa manera, la calificación es CERO.
3. **Todo integrado en la rama principal.** La entrega es un *commit* etiquetado de la rama principal. Si al clonar esa rama falta trabajo que ustedes consideran entregado porque vive en otra rama, la calificación es CERO.
4. **Cada uno responde por lo suyo.** En la defensa, cada integrante tiene que explicar el código y las mediciones que se le atribuyen: por qué están escritas así, qué pasa si se cambia tal cosa, de dónde salió tal número. El contenido que su autor declarado no sepa explicar se retira de la evaluación; si eso ocurre de forma sistemática en un integrante, su calificación es CERO. Esto aplica igual sea el código de un compañero, de un tutorial o de una herramienta automática: lo que se entrega, se entiende.

8 Lista de comprobación antes de entregar

1. Clonar el repositorio en un directorio vacío y compilar el documento siguiendo únicamente lo que dice el README. Si no sale el mismo PDF, todavía no está listo.
2. Ejecutar todas las suites de pruebas sobre ese clon limpio y comprobar que pasan.
3. Ejecutar todas las verificaciones de sumas y comprobar que pasan enteras.
4. Abrir el PDF y comprobar que no hay ninguna referencia ni cita sin resolver.
5. Recorrer las cifras del documento y comprobar, una por una, que cada una se reproduce con el comando que el propio documento indica.
6. Comprobar que ninguna rama va por delante de la principal.
7. Comprobar que el flujo automático está en verde en el *commit* que se entrega.
8. Crear la etiqueta de versión y la publicación con el documento y el instalable.
9. Subir al SGA el PDF con carátula y la dirección del repositorio en una sola línea.

Referencias

- [1] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.
- [2] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [3] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [4] A. Vargha and H. D. Delaney, “A critique and improvement of the CL common language effect size statistics of McGraw and Wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [5] International Organization for Standardization, “ISO/IEC 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model,” standard, ISO, Geneva, Switzerland, 2023.
- [6] Association for Computing Machinery, “ACM code of ethics and professional conduct.” <https://www.acm.org/code-of-ethics>, 2018.
- [7] Y. Hammad, A. Al-Said Ahmad, and P. Andras, “An empirical study on the performance overhead of code instrumentation in containerised microservices,” *Journal of Systems and Software*, vol. 230, p. 112573, 2025.
- [8] S. Henning and W. Hasselbring, “A configurable method for benchmarking scalability of cloud-native applications,” *Empirical Software Engineering*, vol. 27, no. 6, p. 143, 2022.
- [9] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, “Design, monitoring, and testing of microservices systems: The practitioners’ perspective,” *Journal of Systems and Software*, vol. 182, p. 111061, 2021.
- [10] M. Hui, L. Wang, H. Li, R. Yang, Y. Song, H. Zhuang, D. Cui, and Q. Li, “Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,” *Journal of Systems and Software*, vol. 220, p. 112232, 2025.

Anexo. Rúbrica de calificación

La tabla reproduce los entregables de la sección 3 con una columna adicional: el porcentaje de la calificación de la entrega que corresponde a cada uno. Los 29 entregables en **Hecho** siguen en Hecho y no se califican en esta entrega; los 22 restantes reparten el 100 % en proporción a la profundidad, la complejidad, la amplitud y el tiempo de trabajo que exige llevarlos a Hecho, con la calidad descrita en la sección 4, desde el estado en que se encontraban el domingo 6 de septiembre de 2026. Se califica el estado del repositorio en su último *commit* hasta el **viernes 18 de septiembre de 2026 a las 23:55**.

#	Entregable	Ámbito	Estado	% calificación
1	Refactorización en capas y principios SOLID	Arquitectura	Hecho	—
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho	—
3	Vistas de arquitectura C4 (tres niveles)	Arquitectura	Modificar	3,9 %
4	Contrato de interfaz OpenAPI	Arquitectura	Modificar	4,3 %
5	Esquema de base de datos y migraciones	Datos	Culminar	5,9 %
6	Persistencia distribuida operando	Datos	Modificar	3,9 %
7	Aplicaciones web funcionales (cuatro interfaces)	Aplicaciones	Hecho	—
8	Aplicación móvil funcional	Aplicaciones	Hecho	—
9	Capacidades de dispositivo (biometría y notificaciones)	Aplicaciones	Hecho	—
10	Integración web + móvil + backend	Aplicaciones	Hecho	—
11	Pruebas de unidad e integración del backend Java	Pruebas	Hecho	—
12	Pruebas del microservicio en Python con cobertura medida	Pruebas	Modificar	3,1 %
13	Pruebas de contrato verificadas contra servidor real	Pruebas	Hecho	—
14	Pruebas de carga	Pruebas	Hecho	—
15	Pipeline de integración con dependencias declaradas	Flujo	Hecho	—
16	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Hecho	—
17	Procesamiento paralelo	Medición	Hecho	—
18	Bitácora de auditoría encadenada (implementación en Python)	Arquitectura	Hecho	—
19	Protocolo experimental e instrumental de medición	Medición	Hecho	—
20	Datos crudos preservados	Medición	Hecho	—
21	Análisis estadístico con tamaño de efecto e intervalos	Medición	Modificar	5,5 %
22	Evaluación de calidad con las características medidas	Medición	Modificar	5,1 %
23	Manuscrito consolidado, compilable y sin referencias indefinidas	Manuscrito	Hecho	—
24	Bibliografía verificada con identificador persistente	Manuscrito	Hecho	—
25	Preguntas de investigación formuladas y respondidas	Manuscrito	Hecho	—
26	Amenazas a la validez con mitigación declarada	Manuscrito	Hecho	—
27	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho	—
28	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho	—
29	Licencia y archivo de citación	Formalidades	Hecho	—
30	Actas de reunión	Formalidades	Hecho	—

#	Entregable	Ámbito	Estado	% calificación
31	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho	—
32	Capturas de la aplicación en release	Evidencia	Modificar	3,5 %
33	Paquete móvil publicado en release	Aplicaciones	Culminar	3,5 %
34	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Hecho	—
35	Instalación reproducible de todas las interfaces desde un clon limpio	Flujo	Modificar	3,5 %
36	Pruebas de extremo a extremo sobre navegador	Pruebas	Modificar	5,5 %
37	Pruebas de integración sobre contenedores efímeros	Pruebas	Modificar	5,9 %
38	Cobertura reportada con una sola cifra coherente	Pruebas	Culminar	5,5 %
39	Dependencias con versiones existentes e instalables	Flujo	Hecho	—
40	Publicación de todas las imágenes del sistema	Flujo	Modificar	3,1 %
41	Despliegue que falla cuando debe fallar	Flujo	Hecho	—
42	Mecanismo de auditoría único y coherente en todo el sistema	Arquitectura	Modificar	7,5 %
43	Verificador experimental que comprueba la cadena de producción	Medición	Modificar	5,1 %
44	Disparador de inmutabilidad efectivamente aplicado a la base	Datos	Modificar	4,3 %
45	Restricción de privilegios de modificación sobre la bitácora	Datos	Culminar	5,1 %
46	Gestión de secretos fuera del árbol versionado	Formalidades	Culminar	6,3 %
47	Certificado de reproducibilidad consistente con los datos	Medición	Hecho	—
48	Conjunto de resultados de carga único y declarado	Medición	Modificar	4,3 %
49	Fijación de finales de línea para verificación reproducible	Medición	Hecho	—
50	Mapa en el README entre las carpetas del proyecto y la estructura prescrita	Formalidades	Culminar	2,8 %
51	Integración de todas las ramas de trabajo a la principal	Flujo	Culminar	2,4 %
Total (22 entregables calificados)				100,0 %

Guía para culminar el Proyecto Fin de Curso

Equipo FUVV

Aplicaciones Distribuidas (ISR-701) · Séptimo semestre · Carrera de Ingeniería de Software
Facultad de Ciencias de la Computación y Diseño Digital · UTEQ · PPA 2026–2027

El proyecto

SCLI — Sistema de Control de Laboratorios Informáticos

Repositorio: <https://github.com/ffarinangog2/Entrega-final-del-PFC>

Estado revisado sobre la rama `feature/entrega-4`, `commit d0ef60b` del 13 de septiembre de 2026, con 366 `commits` de historia.

Integrantes: Freddy Farinango, Harold Vinueza, Iván Villamarín, Isaías Urbina.

1 Para qué sirve esta guía

Esta guía responde a una sola pregunta: qué les falta para cerrar el Proyecto Fin de Curso con un manuscrito que se sostenga. Para responderla revisé el repositorio entero sobre el último *commit*, entregable por entregable, sin límite de fecha. No me quedé en comprobar que los archivos estuvieran: ejecuté las suites de pruebas, medí yo mismo la cobertura sin los filtros del proyecto, recalculé desde los datos crudos las cifras que el documento publica, ejecuté las verificaciones de sumas contando cuántas pasan y cuántas fallan, y comprobé en cada caso que las pruebas pudieran fallar rompiendo una aserción o el dato.

El resultado es la tabla de la sección 3: los **37 entregables** que el proyecto debía desarrollar, cada uno con su estado. De ellos, 19 están en Hecho y 18 necesitan trabajo. Ese es el alcance real de lo que queda.

Lectura del estado del proyecto

La campaña experimental de este proyecto es la más sólida del curso y no hay que tocarla: recalculé las veinticuatro comparaciones desde los datos crudos y reproducen sin una sola discrepancia. Lo que impide cerrar el PFC es de otra naturaleza. El pipeline analítico de la Unidad 4 compone transformaciones y nunca las ejecuta, de modo que el proyecto no tiene ninguna cifra de aceleramiento que llevar al manuscrito. La cobertura de la aplicación web se sostiene en una prueba de relleno sin aserciones. La bibliografía tiene cinco atribuciones cruzadas. Y el repositorio no tiene licencia, ni etiqueta de versión, ni una sola captura de la aplicación.

2 Los cuatro estados

Estado	Qué significa exactamente
Hecho	El entregable está terminado, es correcto al cien por cien y está listo para usarse en la redacción del manuscrito. Se puede citar tal como está: la cifra que publica se reproduce, el archivo que nombra existe, la prueba que declara puede fallar.
Modificar	El entregable está terminado, pero no está correctamente desarrollado. No está listo para la redacción: usarlo tal como está introduciría en el manuscrito un error o una afirmación que el repositorio no sostiene.
Culminar	El desarrollo está iniciado y hay que continuarlo. La pieza existe y funciona en parte, pero le falta lo que la haría utilizable como evidencia.
Iniciar	El entregable no está presente en el proyecto. Hay que construirlo desde cero.

La diferencia entre Modificar y Culminar, que es la que más se confunde

Un entregable en **Modificar** ya está construido y el trabajo que queda es de corrección: cambiar un número que no cuadra, quitar una afirmación que el código no sostiene, retirar un filtro que infla una medición. Suele costar poco tiempo y suele ser lo que más daño hace si se deja, porque un revisor externo lo detecta abriendo el repositorio y pierde la confianza en todo lo demás.

Un entregable en **Culminar** tiene el trabajo a medias y lo que queda es construcción: ejecutar una campaña que no se ejecutó, escribir el verificador que falta, capturar la evidencia que nadie capturó. Cuesta más tiempo, pero mientras está a medias no engaña a nadie.

Por eso el orden de ataque que propongo más adelante empieza por lo que está en Modificar y no por lo que parece más grande.

3 Todos los entregables, con su estado

La tabla recoge todo lo que el proyecto debía desarrollar, incluido lo que ya está resuelto. La columna de estado es la que manda: lo que aparece en Hecho no hay que volver a tocarlo, y puede citarse en el manuscrito tal como está.

#	Entregable	Ámbito	Estado
1	Refactorización en capas y principios SOLID	Arquitectura	Hecho
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho
3	Vistas de arquitectura C4 (tres niveles) con fuente versionada	Arquitectura	Hecho
4	Esquema de base de datos y semillas	Datos	Hecho
5	Persistencia distribuida operando con evidencia de tolerancia a fallos	Datos	Hecho
6	Aplicación web funcional (construye y ejecuta)	Aplicaciones	Hecho
7	Calidad de la aplicación web (pruebas de unidad)	Pruebas	Modificar
8	Aplicación móvil funcional	Aplicaciones	Hecho
9	Capacidades de dispositivo (cámara y notificaciones)	Aplicaciones	Culminar
10	Integración web + móvil + backend	Aplicaciones	Hecho
11	Pruebas de unidad e integración del backend	Pruebas	Hecho
12	Pruebas de contrato	Pruebas	Hecho
13	Pruebas de extremo a extremo	Pruebas	Hecho

#	Entregable	Ámbito	Estado
14	Pruebas de carga	Pruebas	Hecho
15	Pipeline de integración y despliegue con publicación etiquetada	Flujo	Iniciar
16	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Culminar
17	Procesamiento paralelo	Medición	Culminar
18	Protocolo experimental e instrumental de medición	Medición	Hecho
19	Datos crudos preservados con metadatos y sumas de verificación	Medición	Culminar
20	Fijación de finales de línea para verificación reproducible	Medición	Culminar
21	Manuscrito compilable, acumulativo y sin referencias indefinidas	Manuscrito	Hecho
22	Bibliografía verificada con identificador persistente	Manuscrito	Modificar
23	Amenazas a la validez con mitigación declarada	Manuscrito	Hecho
24	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho
25	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho
26	Licencia y archivo de citación	Formalidades	Iniciar
27	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho
28	Capturas de la aplicación en release	Evidencia	Culminar
29	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Hecho
30	Contrato de interfaz OpenAPI de todos los servicios	Arquitectura	Modificar
31	Escenario de fiabilidad ejecutado	Medición	Modificar
32	Evaluación de calidad con las cinco características medidas	Medición	Modificar
33	Incorporación al documento de toda la evidencia medida	Manuscrito	Modificar
34	Preguntas de investigación formuladas y respondidas	Manuscrito	Modificar
35	Paquete móvil firmado y publicado en release	Aplicaciones	Modificar
36	Actas de reunión	Formalidades	Modificar
37	Documento único por entrega, sin duplicados	Manuscrito	Modificar

Resumen

Estado	Entregables	Proporción
Iniciar	2	5 %
Culminar	6	16 %
Modificar	10	27 %
Hecho	19	51 %
Total	37	100 %

El trabajo pendiente se reparte así por ámbito: Medición (5); Manuscrito (4); Aplicaciones (2); Formalidades (2); Pruebas (1); Flujo (1); Observabilidad (1); Evidencia (1); Arquitectura (1). Esa distribución sirve para repartirse el trabajo sin que dos personas se pisen.

4 El trabajo que queda, entregable por entregable

De cada entregable pendiente doy tres cosas. **La situación** es lo que encontré al revisar, con el archivo y el dato concreto, para que no tengan que adivinar de qué hablo. **Qué falta** es la acción que cierra el punto. Y el **estado** dice de dónde parte cada uno.

Entregables en Iniciar — no existe nada

15. Pipeline de integración y despliegue con publicación etiquetada

Flujo · **Iniciar**

La situación. Comprobé que `git tag -l` no devuelve ninguna etiqueta ni en el clon local ni en el remoto. Leí los tres flujos: `ci-cd.yml` se dispara por envío y por solicitud de incorporación sin ningún filtro de etiquetas, `cd.yml` solo por invocación y `docs.yml` por envío a una rama concreta. Busqué en todo `.github/` las acciones de publicación de versiones y el comando equivalente y no hay ni una coincidencia. El instalable firmado se sube solo como artefacto temporal del flujo, con el nombre del hash del commit, y las imágenes se etiquetan también por hash. No existe ninguna versión publicada que reúna el documento y el instalable.

Qué falta

Crear la etiqueta de versión de la entrega, añadir al flujo el disparador por etiqueta y un paso que publique la versión adjuntando el PDF del manuscrito y el instalable con su suma, y etiquetar las imágenes también con la versión.

26. Licencia y archivo de citación

Formalidades · **Iniciar**

La situación. Busqué en el índice completo de Git cualquier archivo cuyo nombre contenga «licen» o «copying» y no hay ninguna coincidencia. Revisé `CITATION.cff` línea por línea: declara versión, fecha, repositorio y los cuatro autores, pero no tiene campo de licencia. Busqué también en el `README.md` las palabras licencia, license, MIT y Apache y no aparece ninguna. Sin licencia la obra queda como todos los derechos reservados y ningún tercero puede reutilizarla, lo que es incompatible con publicar el artefacto junto al manuscrito.

Qué falta

Crear el archivo `LICENSE` con una licencia abierta reconocida, declarar el mismo identificador en `CITATION.cff` y enunciarlo en el `README`.

Entregables en Culminar — el desarrollo está a medias

9. Capacidades de dispositivo (cámara y notificaciones)

Aplicaciones · **Culminar**

La situación. La cadena de notificaciones está construida en seis pasos, desde el servicio de mensajería hasta el adaptador, de modo que el desarrollo existe. Lo que no existe es la prueba: `docs/evidencias/fcm-e2e.md` declara en su segunda línea «IMPLEMENTADO PARCIALMENTE — VALIDACIÓN E2E NO EJECUTADA» y su propia sección «Evidencia que falta» reconoce que no hay constancia de un evento real recibido en un dispositivo. Contrasté con el `README.md` y coincide: la línea 175 dice que el emisor «no está completamente operativo» y la 372 lo lista como pendiente. Busqué en todo el árbol cualquier captura o traza fechada de una notificación recibida y no hay ninguna.

Qué falta

Configurar el mismo proyecto de mensajería en cliente y servidor con secretos, provocar un evento real, versionar la captura o traza fechada de la notificación recibida en el dispositivo y

actualizar los dos documentos que hoy la declaran pendiente.

16. Observabilidad: métricas, trazas, registros y tableros

Observabilidad · **Culminar**

La situación. Abrí la única captura publicada del tablero, `release/screenshots/panel-monitoreo.png`, y de los seis paneles los tres superiores sí muestran series con datos —tasa de peticiones con tres servicios, latencia en tres percentiles y errores de servidor con dos servicios— pero los tres inferiores siguen rotulados «No data»: el de nodos y rangos de la base distribuida, el de recursos de contenedores y el de latencia extremo a extremo del móvil. La ventana de la captura es de quince minutos y el archivo no se regeneró con el trabajo correctivo, de modo que la mitad del tablero se publica vacía.

Qué falta

Levantar los exportadores que alimentan los tres paneles vacíos, esperar a que haya serie y volver a capturar el tablero con los seis paneles poblados.

17. Procesamiento paralelo

Medición · **Culminar**

La situación. Verifiqué que `spark/pipeline.py` no fue tocado por el trabajo correctivo y sigue sin provocar evaluación alguna: busqué `.write`, `.count()`, `.collect()`, `.show()` y escritura Parquet en todo el archivo y obtuve cero coincidencias, y la propia documentación interna de la función declara «compone las transformaciones sin provocar evaluación ni escritura». Abrí `spark/analisis.ipynb` y conté diez celdas, las diez sin salida, y comprobé además que importan de la línea base en pandas y no del pipeline, de modo que el cuaderno ni siquiera ejercita Spark. No existe ningún archivo de resultados ni ningún guion de medición: busqué «speedup» y «Amdahl» en los tres directorios candidatos sin resultado. El propio `docs/main.tex` lo reconoce al declarar que no presenta ninguna cifra de aceleramiento. A esto se suma que la línea 1182 del mismo documento sigue afirmando que «ambos scripts exportan su resultado en formato Parquet», lo que es falso: solo `spark/baseline.py` exporta.

Qué falta

Añadir una acción terminal al pipeline que fuerce la evaluación y escriba el resultado; ejecutarlo contra el clúster con al menos cinco grados de paralelismo y varias repeticiones por grado; versionar los tiempos crudos, el resumen y la figura; calcular el aceleramiento y ajustar la fracción no paralelizable; y corregir la afirmación del documento sobre la exportación.

19. Datos crudos preservados con metadatos y sumas de verificación

Medición · **Culminar**

La situación. Ejecuté la verificación del manifiesto sobre el árbol limpio de hoy y pasa entera: 468 líneas, 468 correctos y cero fallidos. El fallo anterior está resuelto y el manifiesto está regenerado contra el contenido realmente versionado. Lo que falta es la segunda mitad del criterio: busqué la orden de verificación en todo `.github/` y en `scripts/`, y las tres únicas coincidencias están dentro del trabajo de firma del instalable y se refieren al manifiesto del propio paquete, no al de los resultados experimentales. Ninguno de los tres flujos ejecuta la verificación en cada envío, de modo que nada impide que el manifiesto vuelva a desincronizarse sin que nadie lo note.

Qué falta

Añadir al flujo un trabajo que ejecute la verificación del manifiesto de resultados desde la raíz y falle la construcción si alguna suma no coincide.

20. Fijación de finales de línea para verificación reproducible

Medición · **Culminar**

La situación. Leí `.gitattributes` y sigue teniendo las mismas tres reglas, sin ninguna regla general

de normalización. Para medir el alcance real extraje las 468 rutas del manifiesto y las pasé por la resolución de atributos de Git, que es la comprobación autoritativa: solo 2 rutas reciben finales de Unix fijados y las 466 restantes quedan sin especificar. Clasifiqué después las 468 entradas por tipo y comprobé que 467 son texto, de modo que 465 archivos de texto que el manifiesto hashea siguen sin fijación y su verificación se rompería en un clon de Windows.

Qué falta

Añadir la regla general de normalización al principio de `.gitattributes`, renormalizar el árbol confirmando el cambio en un commit propio, y regenerar el manifiesto sobre el contenido renormalizado.

28. Capturas de la aplicación en release

Evidencia · **Culminar**

La situación. Listé `release/screenshots/` y contiene únicamente el marcador de carpeta y una imagen, que abrí y no es de la aplicación: es el navegador sobre el tablero de observabilidad. Recorrí además todas las imágenes versionadas del repositorio y las que hay son diagramas de arquitectura, evidencias del clúster, gráficas de latencia y capturas de commits; no encontré ni una sola captura de la interfaz web ni de la aplicación móvil, pese a que el instalable firmado sí está publicado. Comprobé también que `docs/main.tex` ya declara esa carpeta en su ruta de gráficos, de modo que el mecanismo existe y lo que falta es el contenido.

Qué falta

Capturar las pantallas principales de la aplicación web y de la móvil en ejecución con datos reales, versionarlas en esa carpeta e insertarlas en el manuscrito desde el párrafo que las interpreta.

Entregables en Modificar — están hechos pero no son utilizables como evidencia

7. Calidad de la aplicación web (pruebas de unidad)

Pruebas · **Modificar**

La situación. Copié `apps/web` fuera del repositorio y ejecuté la suite completa: 41 archivos y 280 pruebas pasan, con 85,96 % de sentencias globales. Aislé la capa de servicios y da 95,16 % de sentencias y 92,17 % de funciones. Después retiré `apps/web/src/services/apisCoverage.test.ts` y volví a ejecutar: quedan 277 pruebas y la capa de servicios cae a 63,31 % de sentencias y 25,22 % de funciones. Abrí ese archivo, que sigue sin tocar: 171 líneas que invocan alrededor de noventa funciones con la capa de red simulada y solo tres aserciones, todas del tipo «fue llamada», sin verificar ruta, método, cuerpo ni mapeo. Verifiqué además el denominador: el reporte cubre 67 archivos y quedan fuera 12 fuentes, entre ellas el componente raíz, el enrutador y tres páginas, porque `apps/web/vite.config.ts` sigue sin activar la medición sobre todos los archivos. Lo que sí resolvieron es que el documento ya no traslada cifras concretas de cobertura web, con lo que no publica una afirmación falsa; pero tampoco puede publicar ninguna.

Qué falta

Dotar a `apisCoverage.test.ts` de aserciones reales sobre ruta, método, cuerpo y forma de la respuesta, o retirarlo; activar la medición sobre todos los archivos fuente en `apps/web/vite.config.ts`; y publicar la cifra resultante aunque sea más baja.

22. Bibliografía verificada con identificador persistente

Manuscrito · **Modificar**

La situación. Leí `docs/main.tex` en las líneas 257 a 266 y las cinco atribuciones cruzadas siguen exactamente igual. Comprobé cada clave contra `docs/Referencias.bib`, que contiene 32 entradas: el texto atribuye el modelo de conjuntos distribuidos a Zaharia, la guía de marcos de datos a Chambers y Zaharia y la revisión de analítica de macrodatos a Salloum, y las tres veces cita una

clave que corresponde a un estudio empírico de otros autores sobre dificultades de desarrollo en Spark; atribuye las guías de experimentación a Wohlin citando una entrada que es en realidad un trabajo sobre amenazas a la validez; y atribuye el diseño experimental a Jain citando una entrada sobre muestreo. Busqué los cinco apellidos en el archivo de bibliografía y ninguno aparece.

Qué falta

Incorporar a la bibliografía las cinco obras reales, con todos sus campos comprobados contra el registro del editor, y sustituir las claves prestadas en las líneas 257 a 266; o reescribir el texto para nombrar al autor que cada clave sí designa.

30. Contrato de interfaz OpenAPI de todos los servicios

Arquitectura · **Modificar**

La situación. Ejecuté las pruebas de los cinco módulos y todas pasan, incluida la que acredita las 77 operaciones con la funcionalidad opcional deshabilitada frente a las 79 habilitables, y las nueve del servicio de puerta de entrada. El verificador nuevo ya no es circular: obtiene las operaciones del registro de rutas que el marco resuelve en tiempo de ejecución, no de un extractor por expresión regular, y las cantidades 12, 44, 71, 79 y 224 están fijadas en las aserciones. Comprobé que las pruebas pueden fallar: al borrar una ruta del contrato de usuarios el módulo se puso en rojo, y al añadir una ruta fantasma al contrato de la puerta de entrada también. El límite es doble: la documentación sigue afirmando que la prueba de la puerta de entrada usa servicios locales efímeros cuando un commit posterior los sustituyó por un doble, de modo que el salto real ya no se ejercita, y la completitud se define desde los contratos de los servicios detrás más alias explícitos, por lo que una ruta aceptada y no documentada por ningún contrato no se detectaría.

Qué falta

Corregir la afirmación de la documentación sobre lo que ejercita la prueba de la puerta de entrada, y cerrar el hueco de completitud comparando también contra las rutas que la puerta acepta de hecho.

31. Escenario de fiabilidad ejecutado

Medición · **Modificar**

La situación. La causa está corregida en el código y no solo en el texto: el guion de carga conserva ahora los dos testigos y su expiración, renueva con treinta segundos de margen y reintenta una sola vez ante un rechazo, y sus cinco pruebas pasan y se ponen en rojo al alterar el cálculo de la expiración. Recaliculé el censo histórico desde los crudos de las diez repeticiones y obtuve 889.868 peticiones, 668.367 rechazos y 612 errores de servidor, es decir 75,1086 %, exactamente lo que declara el documento de cierre. El ensayo correctivo es real: sus marcas dan 1.500 segundos exactos y no hay un solo rechazo en su registro de fallos. Lo que impide cerrar es que la campaña correctiva no se ejecutó: el archivo de resultados es una plantilla con diez filas vacías, el directorio de datos crudos de esa campaña no existe, y pese a ello el dictamen afirma que la corrección se verificó con una repetición completa de una hora. Además los percentiles oficiales del informe siguen saliendo de la campaña contaminada.

Qué falta

Ejecutar de verdad la campaña correctiva de una hora con el guion corregido, versionar sus datos crudos, llevar sus percentiles al documento y reescribir el dictamen para que describa exactamente lo que se ejecutó.

32. Evaluación de calidad con las cinco características medidas

Medición · **Modificar**

La situación. Comprobé una por una las afirmaciones de procedencia y todas se sostienen: los manifiestos de las tres repeticiones de mantenibilidad declaran ventanas de tiempo que no se solapan, los tres informes de cobertura tienen resúmenes distintos que coinciden con los publicados y los

identificadores de sesión del instrumental son también distintos, de modo que son invocaciones separadas y no un archivo triplicado. El texto ya no deduce independencia de una desviación cero y conserva el umbral del 70 % y el incumplimiento de la aplicación móvil en 38,34 %. Regeneré el análisis completo y es idéntico al versionado. El límite lo encontré al ejecutar la verificación de sumas del paquete canónico de evidencia: pasa 7 de 50 y fallan 43, todas por normalización de finales de línea, de modo que ese paquete no es verificable por un tercero tal como se entrega.

Qué falta

Renormalizar el paquete canónico de evidencia y regenerar sus sumas, de forma que la verificación pase entera sobre un clon limpio en cualquier sistema.

33. Incorporación al documento de toda la evidencia medida

Manuscrito · [Modificar](#)

La situación. Los dos defectos anteriores están resueltos y los verifiqué de forma independiente: la verificación del manifiesto general da 468 correctos y 0 incorrectos sobre un archivo de 468 líneas, frente a las 198 entradas que el texto declaraba antes; y reproduce el censo de errores sin usar su resumen, recorriendo los manifiestos crudos y contando 10.000 registros y 948 con código igual o superior a 400, con repartos que coinciden cifra por cifra con la tabla publicada. Comprobé también que cada error conserva sus marcas de tiempo, que las 10.000 latencias coinciden con la resta de sus marcas y que las medias por corrida reproducen las publicadas. Lo que impide cerrar es que la prueba que el propio equipo añadió para blindar esto está en rojo: falla al comparar medias con doce decimales por una diferencia del orden de una parte en diez billones, y como el flujo ejecuta el descubrimiento de pruebas sin condicionales, el trabajo experimental queda fallando en el commit entregado.

Qué falta

Relajar la tolerancia de esa comparación al orden de magnitud que la aritmética de punto flotante permite y dejar el flujo en verde.

34. Preguntas de investigación formuladas y respondidas

Manuscrito · [Modificar](#)

La situación. Recalculé los percentiles desde el dato bruto: leí las ocho filas de la operación declarada en los archivos de las ocho repeticiones y reproduce exactamente media 45,50 ms con intervalo [27,30; 63,70] y media 371,25 ms con intervalo [66,91; 675,59], las mismas cifras que publica hoy el documento. Conté 57.241 consultas y 0 fallos en la población central, cifra que el texto declara literalmente. Comprobé que no tocaron el dato y que los umbrales del prerregistro son anteriores a la campaña, con commits del 18 y 24 de agosto frente a una ejecución del 30. El analizador ya no puede descartar filas en silencio y sus tres pruebas nuevas se ponen en rojo al mutar la condición que excluye la fila agregada. El límite es que el propio documento admite ahora que las dos preguntas se formalizaron después de la ejecución sobre criterios preexistentes, que el cambio de población convierte un percentil que no cumplía en uno que cumple, y que la cola sigue muy dispersa, con desviación de 364,03 ms y un límite superior de 675,59 ms frente a un umbral de 750.

Qué falta

Declarar en el apartado de amenazas que la formalización de las preguntas es posterior a la ejecución, y reportar el percentil con su dispersión real en lugar de presentarlo como cumplimiento holgado.

35. Paquete móvil firmado y publicado en release

Aplicaciones · [Modificar](#)

La situación. Leí el trabajo completo de firma en el flujo y el paso de comprobación termina ahora con un fallo explícito ante la ausencia de cualquiera de los cuatro secretos, sin tolerancia

al error, de modo que la falta de material de firma hace fallar el trabajo en lugar de producir un verde engañoso. Verifiqué que el paquete sigue intacto y firmado: su suma de verificación pasa y al inspeccionar los bytes encontré el bloque de firma con los identificadores de los esquemas v2 y v3 junto a los archivos del esquema v1. El límite es que el trabajo sigue condicionado a envíos sobre una rama concreta, de modo que en cualquier otra no se ejecuta y un flujo en verde continúa sin demostrar que el instalable se generó.

Qué falta

Quitar la restricción de rama del trabajo de firma o, alternativamente, adjuntar el instalable y su suma a la publicación etiquetada de la entrega, de modo que la generación quede acreditada.

36. Actas de reunión

Formalidades · **Modificar**

La situación. La incoherencia cronológica desapareció porque ya no se datan reuniones: los cinco documentos declaran como única fecha verificable la de su consolidación, y comprobé que el commit que los crea está fechado ese mismo día y a esa hora. Extraje los once resúmenes de commit citados y verifiqué uno a uno que todos existen y que sus fechas caen dentro del periodo que cada registro les atribuye. El texto retira explícitamente las afirmaciones que el control de versiones no sostiene. Lo que queda a la vista es que el indicador sigue sin su artefacto central: no hay evidencia contemporánea de ninguna reunión, y la revisión cruzada continúa vacía, con cero revisores solicitados, cero revisiones y cero aprobaciones.

Qué falta

Registrar las próximas reuniones en el momento en que ocurren, y practicar revisión cruzada efectiva: cada solicitud de incorporación revisada y aprobada por un integrante distinto del autor.

37. Documento único por entrega, sin duplicados

Manuscrito · **Modificar**

La situación. Compilé la instantánea de la entrega anterior desde una copia limpia con el guion que añadieron y terminó con código de salida 0, catorce páginas, cero referencias y citas indefinidas y ninguna figura ausente; su verificación de sumas pasa 21 de 21 y su fuente es idéntica a la del commit histórico. Confirmé además que el documento vivo sigue sano, con 33 páginas y cero referencias y citas indefinidas. El defecto persiste intacto en la otra carpeta: el documento de esta entrega sigue incluyendo por referencia las secciones vivas y, compilado hoy, aborta por una figura ausente y deja 21 referencias y 21 citas sin resolver. A ello se suma que el índice de documentación se contradice al describir esa misma carpeta como instantánea histórica dos párrafos después de declarar que las fuentes vigentes están precisamente ahí.

Qué falta

Aplicar a la carpeta de esta entrega el mismo tratamiento que ya funcionó en la anterior: copiar las secciones en lugar de incluirlas por referencia, o dejar solo el PDF con su nota, y corregir el índice de documentación.

5 Por dónde empezar

El orden que sugiero no es el de la tabla, sino el que evita rehacer trabajo.

1. **Primero lo que está en Modificar y afecta al documento.** Son correcciones cortas y son las que un revisor detecta enseguida: una cifra que no cuadra, una afirmación que el código no sostiene, una tabla con filas contradictorias. Cerrarlas primero sube mucho la calidad percibida por poco esfuerzo.

2. **Después lo que cambia números.** Toda corrección de medición obliga a reescribir tablas y a veces conclusiones. Hacerlo antes de redactar ahorra reescribir dos veces.
3. **Luego lo que está en Culminar y en Iniciar.** Es lo que más tiempo cuesta, así que conviene repartirlo por ámbito entre los cuatro y empezarlo en paralelo con lo anterior.
4. **Al final la redacción, las capturas y la publicación etiquetada.** Dependen de que los números ya sean los definitivos.
5. **Y desde el primer día, la integración.** Nada de acumular ramas. Cada cosa terminada se integra a la rama principal por solicitud de incorporación revisada por un compañero distinto del autor. Dejar la integración para el último día es la forma más frecuente de perder trabajo ya hecho.

6 Del repositorio al manuscrito

Lo que ustedes tienen en el repositorio no es el manuscrito: es el material del que sale el manuscrito. Esta sección explica cómo se traduce lo uno en lo otro, porque es donde los equipos suelen perder el trabajo que ya hicieron.

La regla que ordena todo

Cada afirmación no trivial del documento tiene que poder comprobarse abriendo el repositorio. Esa es la única regla, y de ella se derivan las demás. Si el documento dice que la cobertura es del ochenta por ciento, tiene que existir el comando que produce ese ochenta por ciento. Si dice que una campaña se ejecutó diez veces, tienen que estar los diez directorios de datos crudos. Si dice que un umbral se fijó antes de medir, la fecha del *commit* que lo fija tiene que ser anterior a la fecha de la ejecución.

Un evaluador externo no comprueba si ustedes trabajaron: comprueba si lo que afirman se sostiene. Y una afirmación que no se sostiene hace más daño que una ausencia reconocida, porque la ausencia es honesta y la afirmación falsa obliga al lector a desconfiar de todo lo demás. Los estándares de comunicación empírica en ingeniería de software insisten en este punto por encima de cualquier otro [1, 2].

De qué entregable sale cada parte del documento

Parte del manuscrito	De dónde sale
Contexto y problema	Los registros de decisión de arquitectura y el análisis del dominio. El problema tiene que estar cuantificado, no enunciado.
Arquitectura del sistema	Las vistas C4 en sus tres niveles, con la fuente versionada, y el contrato de interfaz. Cada tecnología que el diagrama nombra debe aparecer en el archivo de dependencias del componente al que se atribuye.
Datos y consistencia	El esquema, las migraciones y la evidencia de fragmentación, replicación y tolerancia a fallos. La posición en el teorema CAP se argumenta por agregado del dominio, no para todo el sistema.
Procesamiento paralelo	El trabajo distribuido con su curva de aceleramiento y el ajuste de la fracción no paralelizable. Si el resultado es una desaceleración, se reporta como tal y se explica.
Diseño experimental	El protocolo, los umbrales prerregistrados y el instrumental de medición. Los umbrales se declaran antes de medir y la fecha del <i>commit</i> lo demuestra [1].
Resultados y análisis	Los datos crudos, el análisis estadístico y las figuras. En comparaciones de rendimiento se usan pruebas no paramétricas y se reporta el tamaño del efecto junto al valor de significación, nunca el valor de significación solo [3, 4].
Evaluación de calidad	Las características medidas con su valor, su intervalo y su umbral, según el modelo de calidad de producto vigente [5]. Una característica que no cumple se consigna como no cumplida.
Amenazas a la validez	Todo lo que quedó en Modificar o Culminar y no se pudo cerrar, dicho con sus palabras y con su alcance real.
Ética	La reflexión referida al código de ética profesional [6], con el tratamiento de los datos personales que el sistema maneja.

Cinco errores que un revisor detecta en la primera lectura

1. **La medición que elige su propio alcance favorable.** Publicar una cobertura calculada sobre los archivos que las pruebas ya tocan, o una compuerta aplicada a las clases que ya pasan. La cifra sube y deja de describir el proyecto. Si hace falta excluir algo, se excluye y se justifica cada exclusión una por una.
2. **El número que no se reproduce.** Una cifra escrita a mano en una tabla se desactualiza en cuanto el código cambia. Toda tabla de resultados debería salir de un guion que lea el dato crudo y emita el bloque del documento.
3. **La prueba que no puede fallar.** Una prueba que pasa igual con el sistema apagado, con la aserción invertida o con la línea que debía verificar borrada, no prueba nada. Compruébenlo ustedes antes de publicarla: rompan algo y miren si se pone en rojo.
4. **El instrumento que altera lo que mide.** La instrumentación tiene coste, y ese coste entra en las cifras que se publican [7]. Conviene declarar en qué condiciones se midió y con qué instrumentación activa.
5. **El documento que va por detrás del repositorio.** El PDF versionado se desfasa del archivo fuente en cuanto alguien corrige el fuente y no recompila. Un trabajo del flujo automático que compile y compare cierra ese agujero para siempre.

Sobre la medición de un sistema distribuido

Medir un sistema de microservicios tiene dificultades propias que conviene nombrar en el documento en lugar de esquivarlas. La carga que se aplica debe ejercitar el camino crítico y no solo las lecturas públicas, porque son las operaciones que escriben las que entran en conflicto. La escalabilidad se mide con un método declarado y repetible, no con una corrida suelta [8]. Y las dificultades que los propios profesionales reportan al probar y monitorizar estos sistemas —la dependencia del entorno, el coste de levantar todo para una prueba, la fragilidad de las pruebas de extremo a extremo— son un marco legítimo para discutir las limitaciones del trabajo [9, 10].

7 Criterios de piso

Estos criterios no puntúan: se comprueban antes de calificar, y el incumplimiento de cualquiera de ellos deja la calificación en **CERO**.

Criterios de piso

1. **El PDF en el SGA con la dirección del repositorio.** Cada estudiante sube al SGA un documento PDF con carátula de identificación que muestre, en una sola línea y de forma claramente legible, la dirección del repositorio donde está el trabajo. Si esa dirección está mal escrita, el repositorio no es accesible, o no se cumple esta directriz, la calificación es CERO.
2. **El PDF se regenera desde el LaTeX.** Clono el repositorio, compilo el archivo principal y tiene que salir el mismo PDF. Las instrucciones de compilación —compilador, orden de comandos, archivo principal y paquetes necesarios— deben estar escritas en el README. Si el PDF no se genera de esa manera, la calificación es CERO.
3. **Todo integrado en la rama principal.** La entrega es un *commit* etiquetado de la rama principal. Si al clonar esa rama falta trabajo que ustedes consideran entregado porque vive en otra rama, la calificación es CERO.
4. **Cada uno responde por lo suyo.** En la defensa, cada integrante tiene que explicar el código y las mediciones que se le atribuyen: por qué están escritas así, qué pasa si se cambia tal cosa, de dónde salió tal número. El contenido que su autor declarado no sepa explicar se retira de la evaluación; si eso ocurre de forma sistemática en un integrante, su calificación es CERO. Esto aplica igual sea el código de un compañero, de un tutorial o de una herramienta automática: lo que se entrega, se entiende.

8 Lista de comprobación antes de entregar

1. Clonar el repositorio en un directorio vacío y compilar el documento siguiendo únicamente lo que dice el README. Si no sale el mismo PDF, todavía no está listo.
2. Ejecutar todas las suites de pruebas sobre ese clon limpio y comprobar que pasan.
3. Ejecutar todas las verificaciones de sumas y comprobar que pasan enteras.
4. Abrir el PDF y comprobar que no hay ninguna referencia ni cita sin resolver.
5. Recorrer las cifras del documento y comprobar, una por una, que cada una se reproduce con el comando que el propio documento indica.
6. Comprobar que ninguna rama va por delante de la principal.
7. Comprobar que el flujo automático está en verde en el *commit* que se entrega.

8. Crear la etiqueta de versión y la publicación con el documento y el instalable.
9. Subir al SGA el PDF con carátula y la dirección del repositorio en una sola línea.

Referencias

- [1] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.
- [2] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [3] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [4] A. Vargha and H. D. Delaney, “A critique and improvement of the CL common language effect size statistics of McGraw and Wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [5] International Organization for Standardization, “ISO/IEC 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model,” standard, ISO, Geneva, Switzerland, 2023.
- [6] Association for Computing Machinery, “ACM code of ethics and professional conduct.” <https://www.acm.org/code-of-ethics>, 2018.
- [7] Y. Hammad, A. Al-Said Ahmad, and P. Andras, “An empirical study on the performance overhead of code instrumentation in containerised microservices,” *Journal of Systems and Software*, vol. 230, p. 112573, 2025.
- [8] S. Henning and W. Hasselbring, “A configurable method for benchmarking scalability of cloud-native applications,” *Empirical Software Engineering*, vol. 27, no. 6, p. 143, 2022.
- [9] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, “Design, monitoring, and testing of microservices systems: The practitioners’ perspective,” *Journal of Systems and Software*, vol. 182, p. 111061, 2021.
- [10] M. Hui, L. Wang, H. Li, R. Yang, Y. Song, H. Zhuang, D. Cui, and Q. Li, “Unveiling the microservices testing methods, challenges, solutions, and solutions gaps: A systematic mapping study,” *Journal of Systems and Software*, vol. 220, p. 112232, 2025.

Anexo. Rúbrica de calificación

La tabla reproduce los entregables de la sección 3 con una columna adicional: el porcentaje de la calificación de la entrega que corresponde a cada uno. Los 19 entregables en **Hecho** siguen en Hecho y no se califican en esta entrega; los 18 restantes reparten el 100 % en proporción a la profundidad, la complejidad, la amplitud y el tiempo de trabajo que exige llevarlos a Hecho, con la calidad descrita en la sección 4, desde el estado en que se encontraban el domingo 6 de septiembre de 2026. Se califica el estado del repositorio en su último *commit* hasta el **viernes 18 de septiembre de 2026 a las 23:55**.

#	Entregable	Ámbito	Estado	% calificación
1	Refactorización en capas y principios SOLID	Arquitectura	Hecho	—
2	Patrones de diseño documentados en registros de decisión	Arquitectura	Hecho	—
3	Vistas de arquitectura C4 (tres niveles) con fuente versionada	Arquitectura	Hecho	—
4	Esquema de base de datos y semillas	Datos	Hecho	—
5	Persistencia distribuida operando con evidencia de tolerancia a fallos	Datos	Hecho	—
6	Aplicación web funcional (construye y ejecuta)	Aplicaciones	Hecho	—
7	Calidad de la aplicación web (pruebas de unidad)	Pruebas	Modificar	5,2 %
8	Aplicación móvil funcional	Aplicaciones	Hecho	—
9	Capacidades de dispositivo (cámara y notificaciones)	Aplicaciones	Culminar	5,7 %
10	Integración web + móvil + backend	Aplicaciones	Hecho	—
11	Pruebas de unidad e integración del backend	Pruebas	Hecho	—
12	Pruebas de contrato	Pruebas	Hecho	—
13	Pruebas de extremo a extremo	Pruebas	Hecho	—
14	Pruebas de carga	Pruebas	Hecho	—
15	Pipeline de integración y despliegue con publicación etiquetada	Flujo	Iniciar	4,7 %
16	Observabilidad: métricas, trazas, registros y tableros	Observabilidad	Culminar	6,2 %
17	Procesamiento paralelo	Medición	Culminar	8,2 %
18	Protocolo experimental e instrumental de medición	Medición	Hecho	—
19	Datos crudos preservados con metadatos y sumas de verificación	Medición	Culminar	5,2 %
20	Fijación de finales de línea para verificación reproducible	Medición	Culminar	4,7 %
21	Manuscrito compilable, acumulativo y sin referencias indefinidas	Manuscrito	Hecho	—
22	Bibliografía verificada con identificador persistente	Manuscrito	Modificar	4,7 %
23	Amenazas a la validez con mitigación declarada	Manuscrito	Hecho	—
24	Reflexión ética referida al código de ética profesional	Manuscrito	Hecho	—
25	README con instrucciones de compilación y reproducibilidad	Formalidades	Hecho	—
26	Licencia y archivo de citación	Formalidades	Iniciar	1,9 %
27	Declaración de uso de herramientas de inteligencia artificial	Formalidades	Hecho	—
28	Capturas de la aplicación en release	Evidencia	Culminar	4,3 %
29	Higiene del repositorio (sin artefactos de compilación versionados)	Formalidades	Hecho	—

#	Entregable	Ámbito	Estado	% calificación
30	Contrato de interfaz OpenAPI de todos los servicios	Arquitectura	Modificar	7,1 %
31	Escenario de fiabilidad ejecutado	Medición	Modificar	6,6 %
32	Evaluación de calidad con las cinco características medidas	Medición	Modificar	8,1 %
33	Incorporación al documento de toda la evidencia medida	Manuscrito	Modificar	7,1 %
34	Preguntas de investigación formuladas y respondidas	Manuscrito	Modificar	5,2 %
35	Paquete móvil firmado y publicado en release	Aplicaciones	Modificar	4,7 %
36	Actas de reunión	Formalidades	Modificar	3,8 %
37	Documento único por entrega, sin duplicados	Manuscrito	Modificar	6,6 %
Total (18 entregables calificados)				100,0 %